

S O U A L C L A R A

D É V E L O P P E U R W E B
E T W E B M O B I L E

D O S S I E R
P R O J E T

J U I N - J U I L L E T

2 0 2 6

Introduction

EcoRide est une startup française qui souhaite réduire l'impact environnemental des déplacements en favorisant le covoiturage. Elle a pour objectif de proposer une plateforme web permettant de mettre en relation des conducteurs et des passagers pour des trajets communs, de manière sécurisée et accessible.

Le directeur technique, José, m'a sollicitée et choisie pour développer cette solution, afin de créer une application fonctionnelle et fiable répondant aux besoins des utilisateurs.

Le projet a été réalisé en autonomie sur une durée de 1 an, à raison de 10 à 15 heures par semaine, en utilisant HTML5, CSS3, JavaScript pour le front-end, PHP8 pour le back-end, MySQL pour la base de données relationnelle, des fichiers JSON pour la base de données non relationnelle, et Git et GitHub pour le versioning.

L'environnement local de développement est XAMPP.

L'objectif principal de la plateforme est double :

- Optimiser l'utilisation des véhicules pour réduire l'empreinte carbone
- Offrir une solution de transport économique aux utilisateurs, avec un système de crédits intégré

Cette mise en place vise à créer une solution complète, fonctionnelle et évolutive, capable de répondre aux besoins des utilisateurs tout en respectant les bonnes pratiques de développement web.

Plan

I. <u>Analyse</u>	5
1. Approche du projet	5
1.1 <i>Choix du projet ECF</i>	5
1.2 <i>Démarche centrée utilisateur</i>	5
2. Contexte	5
2.1 <i>Présentation de la startup EcoRide</i>	5
2.2 <i>Ambition et portée du projet</i>	6
2.3 <i>Expérience personnelle et motivations</i>	6
3. Utilisateurs et personas	7
3.1 <i>Définitions des personas</i>	7
3.2 <i>Méthode et objectifs</i>	7
3.3 <i>Tableau récapitulatif des personas</i>	9
4. Conversion des besoins non techniques en besoin techniques	13
4.1 <i>Méthodologie</i>	13
4.2 <i>Traitement par technologie</i>	13
4.3 <i>Tableau de conversion besoins → fonctionnalités</i>	14
5. Compréhension du projet et gestion	15
6. Users stories et maquettes	16
6.1 <i>Users stories</i>	16
6.2 <i>Réalisation des maquettes</i>	17

6.2.1	Maquette "Index"	
6.2.2	Maquette "Proposer un trajet"	
6.2.3	Maquette "Résultat de recherche"	
7.	Diagrammes de gestion de projet	22
7.1	Diagramme de cas d'utilisation	22
7.2	Diagramme de séquence - Connexion utilisateur	23
7.3	Diagramme de classe (UML)	23
8.	Organisation et suivi du projet	24
8.1	Trello	24
8.2	Git	24
8.3	To-do lists	25
II.	<u>Développement</u>	26
1.	Front-end	26
1.1	HTML - Structure des pages	26
1.1.1	Organisation générale	
1.1.2	Pages principales	
1.1.2.1	Page "Index"	
1.1.2.2	Page "Proposer un trajet"	
1.1.2.3	Page "Résultat de recherche"	
1.1.3	Pages secondaires	
1.2	CSS - Design, styles et responsive	33
1.3	Javascript - Interactions	37

2. Back-end	40
2.1 <i>PHP - Logique métier</i>	41
2.2 <i>Composants réutilisables</i>	42
2.3 <i>Sécurité et validation</i>	43
2.4 <i>Dashboard admin</i>	45
2.5 <i>Dashboard employé</i>	45
3. Fonctionnalités détaillées	46
4. Git et versioning	52
III. <u>Base de données</u>	55
1. Modélisation relationnelle	55
2. Jeu de données	56
3. NoSQL	56
IV. <u>Déploiement</u>	59
1. Préparation de l'environnement	59
1.1 <i>Configuration serveur</i>	
2. Procédure de déploiement	61
2.1 <i>Transfert des fichiers</i>	
2.2 <i>Importation de la base de données</i>	
3. Tests post-déploiement	61
V. Conclusion : évolutions et perspectives du projet	63
Annexes	65

I. Analyse

1. Approche du projet

1.1 *Choix du projet ECF*

J'ai choisi de développer le projet ECF car il représentait à ce moment-là l'opportunité la plus complète de mettre en pratique l'ensemble des compétences que j'étais en train d'acquérir durant ma formation. Il s'agit d'un projet intégrant à la fois le front-end et le back-end, simulant des conditions réelles de développement d'une application web destinée à de véritables utilisateurs.

Ce projet m'a permis de travailler dans une logique professionnelle globale : analyse du besoin, conception UX/UI, développement technique, gestion de base de données et mise en œuvre des bonnes pratiques de sécurité.

Ce qui m'a motivée, c'est que ce type de plateforme m'était utile personnellement et que j'avais déjà en tête la logique du site.

1.2 *Démarche centrée utilisateur*

Cette expérience personnelle m'a permis d'adopter une démarche centrée utilisateur, en anticipant les attentes concrètes et les éventuels points de friction.

Le public visé par la plateforme EcoRide correspond aux *voyageurs soucieux de l'environnement et à ceux recherchant une solution économique pour leurs déplacements*.

J'ai souhaité rendre la plateforme accessible au plus grand nombre en privilégiant une navigation simple et intuitive, adaptée également aux personnes moins à l'aise avec les outils numériques.

2. Contexte

2.1 Présentation de la startup EcoRide

EcoRide est une startup française dont l'objectif est de réduire l'impact environnemental des déplacements en encourageant le covoiturage. Portée par José, son directeur, elle souhaite se faire connaître via une application web dédiée à la mise en relation de conducteurs et de passagers.

José m'a contactée afin de concrétiser ce projet. Suite à un entretien, j'ai été sélectionnée pour concevoir et développer la plateforme EcoRide de A à Z.

2.2 *Ambition et portée du projet*

L'ambition du projet EcoRide est de proposer une plateforme de covoiturage permettant de mettre en relation des conducteurs et des passagers souhaitant effectuer des trajets communs tout en réduisant leur impact environnemental. L'objectif principal est de favoriser une mobilité plus écologique en optimisant l'utilisation des véhicules et en limitant le nombre de voitures en circulation.

Au-delà de l'aspect environnemental, la plateforme vise également à offrir une solution de transport plus économique pour les utilisateurs, avec un système de crédits, sans frais réels à déboursier pour les passagers. Cette approche permet ainsi de répondre à un double enjeu : réduire l'empreinte carbone des déplacements tout en facilitant l'accès à des solutions de transport abordables.

Dans le cadre de ce projet, l'application web doit permettre aux utilisateurs de créer un compte, de proposer ou rechercher des trajets, ainsi que d'interagir avec la plateforme de manière simple et sécurisée. L'objectif est donc de concevoir une solution fonctionnelle, accessible et fiable, répondant aux besoins des utilisateurs tout en respectant les bonnes pratiques de développement web.

2.3 *Expérience personnelle et motivations*

Comme dit précédemment, je me suis sentie proche du projet car je pratique régulièrement le covoiturage et je conduis fréquemment, ce qui m'a permis de constater la

densité de circulation, notamment à Bordeaux, ainsi que les problématiques environnementales associées.

Cette expérience personnelle m’a permis d’identifier certains besoins concrets : la nécessité de trouver rapidement un trajet fiable, de comprendre immédiatement les informations affichées et de pouvoir réserver en toute confiance.

J’ai voulu concevoir un site simple d’accès pour que les utilisateurs ne perçoivent pas l’écologie comme quelque chose de compliqué ou contraignant, mais comme une solution pratique et accessible au quotidien.

L’enjeu était donc de créer une plateforme à la fois engagée sur le plan environnemental et performante sur le plan ergonomique, afin de rendre le covoiturage accessible et attractif pour tous.

3. Utilisateurs et personas

3.1 *Définitions des personas*

Pour concevoir un site adapté aux besoins réels des utilisateurs, j’ai défini plusieurs profils représentatifs : Nino, Finn, Antoine, Théo, Jeanne et Nina.

Ces profils illustrent plusieurs types d’utilisateurs de la plateforme : conducteurs réguliers, passagers occasionnels, étudiants, jeunes actifs, retraités et personnes moins à l’aise avec le numérique.

La création de ces personas repose sur une démarche de conception centrée utilisateur. L’objectif est de représenter des profils réalistes, afin de mieux comprendre les attentes, les contraintes et les comportements des futurs utilisateurs de la plateforme.

3.2 *Méthode et objectifs*

La méthode des personas permet de représenter des utilisateurs fictifs mais réalistes, construits à partir d’objectifs, de comportements et de contraintes identifiés lors de l’analyse du projet.

Cette démarche m’a aidée à éviter une conception centrée uniquement sur mon propre usage, et à prendre en compte des profils variés, avec des attentes et des niveaux d’aisance numérique différents.

L’objectif était d’anticiper les besoins fonctionnels (rechercher, proposer, réserver un trajet) mais également les besoins non fonctionnels tels que la simplicité, la lisibilité et le sentiment de sécurité.

Les personas ont été créés en me mettant à la place des utilisateurs potentiels, en m’appuyant sur mon expérience personnelle du covoiturage et sur l’observation des plateformes existantes.

Pour les définir, j’ai suivi plusieurs étapes :

- Identification des profils d’utilisateurs potentiels de la plateforme (passagers, conducteurs, utilisateurs réguliers ou occasionnels).
- Définition de caractéristiques représentatives (âge, situation professionnelle, habitudes de déplacement, etc.).
- Identification des objectifs principaux liés à l’utilisation du site (protection de l’environnement, réserver facilement, etc.).
- Identification des contraintes et de leurs attentes (notamment en matière de simplicité d’utilisation, de sécurité et de confiance).

3.3 Tableau récapitulatif des personas

Prénom	Rôle	Description	Véhicule(s)
<i>Nino</i>	Passager - conducteur	26 ans, habite à Gradignan. Fait du covoiturage chaque week-end.	Honda Civic, bleu, électrique.
<i>Finn</i>	Passager - conducteur	34 ans, travaille à Toulouse.	Opel Agila, blanche, essence.
<i>Antoine</i>	Passager - conducteur	41 ans, voyage souvent entre Agen et Pau.	Citroën C3, noir, diesel.
<i>Nina</i>	Passagère	24 ans, doctorante à Bordeaux.	-
<i>Théo</i>	Passager	29 ans, technicien à Bayonne.	-
<i>Jeanne</i>	Conductrice	61 ans, vit à Périgueux.	Peugeot 307 gris, essence.

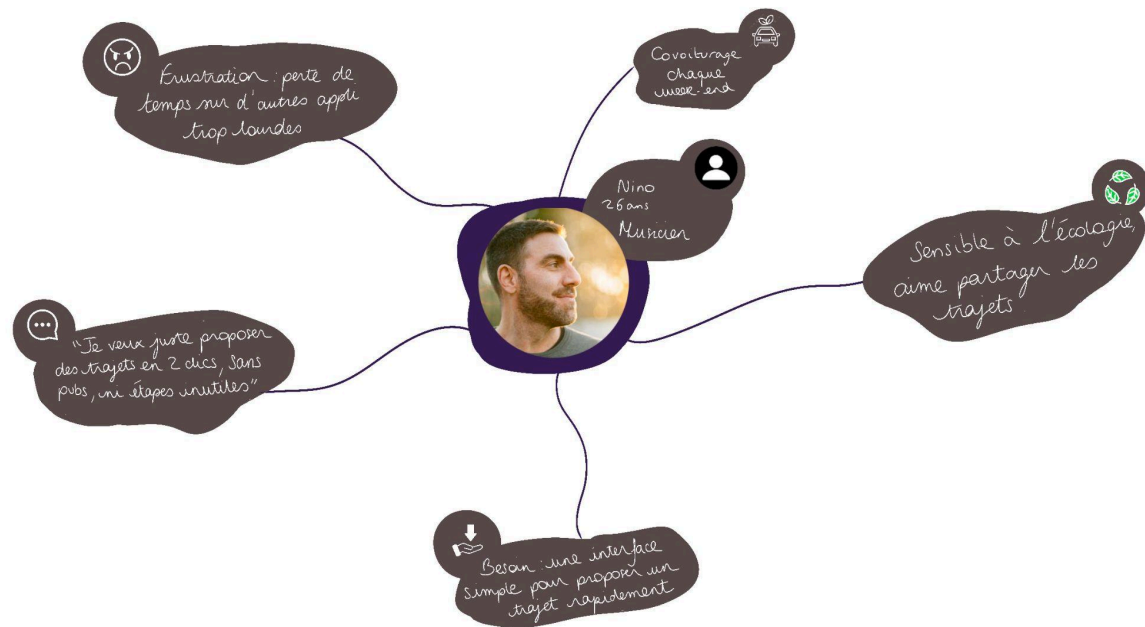
À partir de l'analyse des 6 personas précédents, j'ai identifié et regroupé les principaux besoins des utilisateurs afin de mieux orienter la conception de l'interface et des fonctionnalités du site.

Les schémas ci-dessous permettent de distinguer les besoins fonctionnels, directement liés à l'utilisation du site, et des besoins non fonctionnels, concernant la performance, la sécurité, la simplicité d'usage et la confiance des utilisateurs.

Pour illustrer cette analyse, j'ai choisi de représenter les besoins de trois personas représentatifs, sélectionnés pour leur diversité : Nino, Nina, et Jeanne.

Schémas des besoins fonctionnels et non fonctionnels

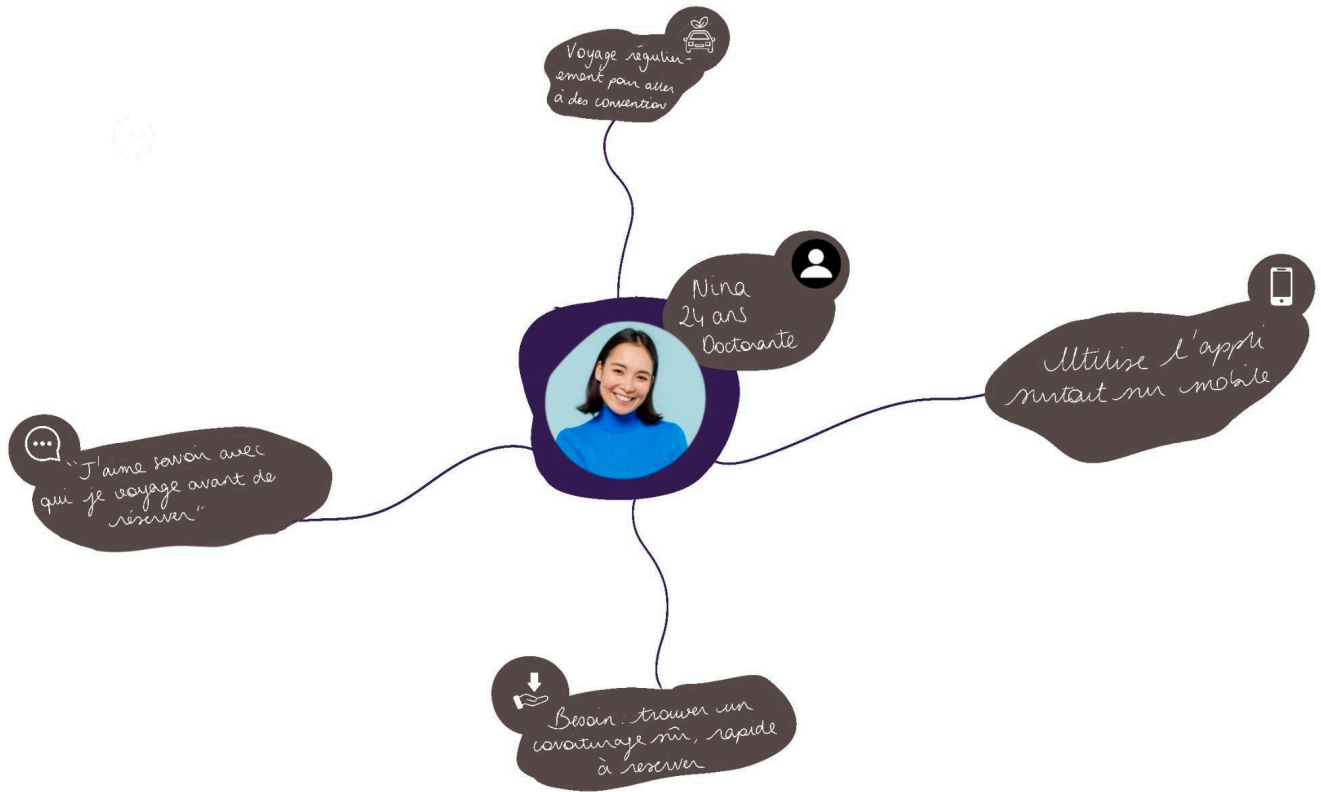
Schéma des besoins du persona Nino



Le persona de *Nino* met en évidence :

- Besoin fonctionnel clair : proposer un trajet rapidement et simplement
- Besoin non fonctionnel : une interface claire, intuitive et sans surcharge visuelle, facilitant l'expérience utilisateur.

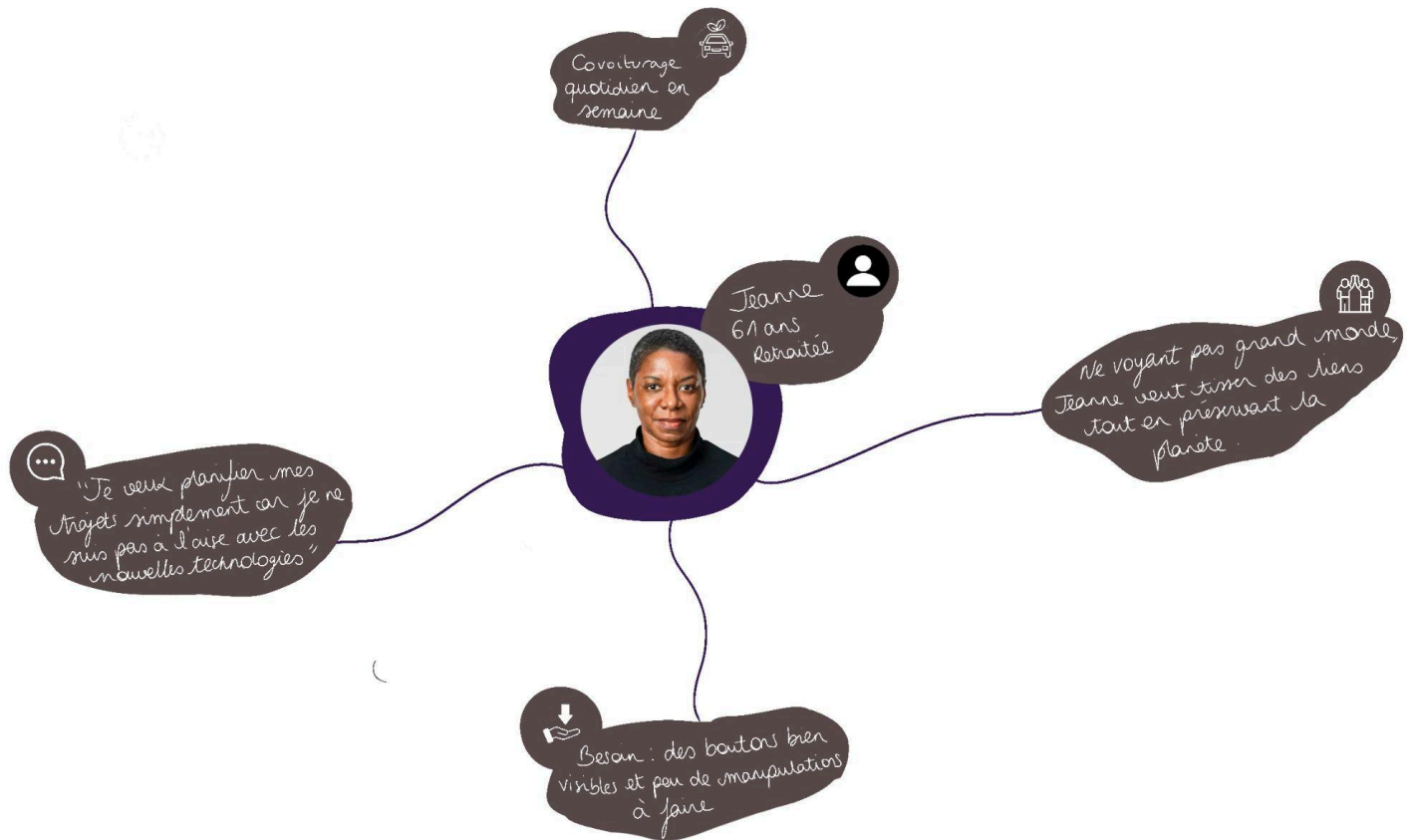
Schéma des besoins du persona Nina



Le persona de *Nina* met en évidence :

- Besoin fonctionnel clair : pouvoir utiliser l'application sur mobile pour rechercher ou réserver un trajet
- Besoin non fonctionnel : la plateforme doit inspirer confiance, avec des informations fiables et un fonctionnement sécurisé

Schéma des besoins du persona Jeanne



Le persona de *Jeanne* met en évidence :

- Besoin fonctionnel clair : des boutons visibles et une navigation simple avec un minimum de manipulations
- Besoin non fonctionnel : créer un lien humain, rassurer et faciliter la confiance dans l'utilisation du site

4. Conversion des besoins non technique en besoin technique

4.1 Méthodologie

Pour concevoir une plateforme répondant aux attentes des utilisateurs, il est essentiel de transformer leurs besoins non techniques, c'est-à-dire ce qu'ils souhaitent ou attendent du site, en besoins techniques concrets, correspondant aux fonctionnalités que le site peut réellement fournir.

Cette étape permet de s'assurer que chaque fonctionnalité développée répond à un besoin précis et améliore l'expérience utilisateur.

Pour convertir les besoins non techniques en besoins techniques, j'ai suivi les étapes suivantes :

1. Identification des besoins utilisateurs
2. Classification en besoins fonctionnels et non fonctionnels
3. Traduction en fonctionnalité technique
4. Répartition par technologie

Cette méthodologie permet de garantir que tous les besoins utilisateurs sont traduits en fonctionnalités réalisables et techniquement cohérentes.

4.2 Traitement par technologie

Pour chaque besoin technique identifié, il est important de préciser où et comment il sera géré dans l'architecture de la plateforme :

- Front-end : gestion de l'interface utilisateur, affichage des données, interaction et ergonomie (formulaires, boutons, etc.) - en HTML, CSS et JS.
- Back-end : traitement des données, logique métier, gestion des bases de données, sécurité et flux applicatifs - en PHP
- Base de données - en MySQL et NoSQL

Cette distinction permet de clarifier la responsabilité technique de chaque fonctionnalité et de s'assurer que les besoins utilisateurs sont correctement pris en compte dans le développement.

4.3 *Tableau de conversion besoins → fonctionnalités*

Le tableau ci-dessous illustre la conversion des besoins en fonctionnalités pour la plateforme EcoRide, en précisant également si chaque besoin est fonctionnel ou non fonctionnel.

Tableau de conversion des besoins non techniques en besoin technique

Type	Besoin non technique	Fonctionnalité technique	Technologie
Fonctionnel	Je veux proposer un trajet facilement	Formulaire simple pour créer un trajet et l'envoyer en BDD	Front-end + PHP + Back-end
	Je veux trouver un covoiturage rapidement	Recherche de trajet avec filtres (date, ville, nombre de passagers)	Front-end + PHP + Back-end
	Je veux savoir avec qui je voyage	Affichage du profil du conducteur/passager avant la réservation	Front-end + PHP + Back-end
	Je veux réserver un trajet sans me tromper	Système de réservation sécurisé avec confirmation	Front-end + PHP + Back-end
	Je veux gérer mes trajets	Tableau de bord pour	Front-end + Back-end

	facilement	gérer ses trajets	
Non fonctionnel	Je veux que l'interface soit claire	Interface intuitive, épurée, et boutons et menus visibles	Front-end
	Je veux que l'appli soit rapide	Page qui se charge facilement, filtrage optimisé	Back-end + Front-end
	Je veux pouvoir utiliser le site sur mobile	Design responsive, adaptation sur mobile et tablette	Front-end
	Je veux être rassuré sur la sécurité des données	Protection des données utilisateurs (chiffrement, HTTPS)	Back-end + PHP

5. Compréhension du projet et gestion

Analyse approfondie

J'ai réalisé une analyse approfondie du projet afin de bien cerner les besoins et attentes des utilisateurs.

Pour appréhender les attentes des utilisateurs, je me suis mise à leur place et j'ai imaginé leurs besoins potentiels, en m'inspirant également de mon entourage, qui présentait une diversité d'âges, de revenus, de lieux de vie, etc. Cette approche, bien que qualitative et basée sur l'expérience, m'a permis d'identifier les besoins fonctionnels (proposer, rechercher, réserver un trajet) ainsi que les besoins non fonctionnels (ergonomie, rapidité, sécurité) à intégrer dans la conception de la plateforme.

6. Users stories et maquettes

6.1 Users stories

Pour faciliter le suivi et l'organisation, le projet a été découpé en petites tâches, associées à des users stories.

Pour structurer le travail, j'ai commencé par définir le *"Qui ? Quoi ? Pourquoi ?"* pour chaque User Story (US) de l'ECF, afin que chaque page et fonctionnalité réponde précisément aux besoins des utilisateurs ciblés.

Les US ont servi de fil conducteur pendant tout le développement, en s'assurant que chaque maquette, puis chaque implémentation technique, corresponde aux besoins exprimés dans les personas.

Extrait du fichier texte du "Qui ? Quoi ? Pourquoi ?" :

" Définition du Projet

Général :

Pour qui ?

« Pour les utilisateurs soucieux de l'environnement et qui recherchent une solution économique pour leurs déplacements » qui veulent uniquement se déplacer en voiture

Quoi ?

Une application écologique avec des thèmes et des couleurs éco responsables

Pourquoi ?

Pour prendre soin de la planète en faisant du covoiturage

US 1 :

Pour qui ?

Pour les visiteurs du site, qui n'ont pas encore de compte et qui découvrent EcoRide

Quoi ?

La page d'accueil présentant l'entreprise, permettant de rechercher un covoiturage avec une barre de recherche

Pourquoi ?

- Pour présenter l'entreprise, ses valeurs
- Pour permettre aux visiteurs de rechercher un covoiturage
- Contacter l'entreprise avec le mail en bas
- Consulter les mentions légales en bas aussi "

J'ai ensuite nommé les différentes pages en associant les US correspondantes. Par exemple :

- Page d'accueil :
 - US 1 : Présentation, barre de recherche, mentions légales, contact
 - US 2 : Menu global, header (présent sur toutes les pages)

J'ai également décrit en détail le contenu de chaque maquette.

Par exemple :

- Maquette 1 : Page d'accueil (US1, US2)
 - Header avec le nom et logo à gauche, sections "Accueil", "Proposer un covoiturage", "Rechercher un covoiturage", "Contact" et photo de profil cliquable à droite pour accéder aux données personnelles, trajets, crédits.
 - Image illustrant EcoRide avec slogan par dessus
 - Ligne de recherche de covoiturage, avec départ, arrivée, date et nombre de passagers (formulaire) + loupe pour lancer la recherche.
 - Présentation à gauche d'une photo du fondateur, à droite présentation d'EcoRide, statistiques, différences avec d'autres plateformes, fonctionnement
 - Footer avec mentions légales, règlement de la plateforme, moyen de contact

Cette méthode a permis de visualiser concrètement chaque page et de vérifier que toutes les US étaient correctement prises en compte avant le développement.

6.2 *Réalisation des maquettes*

Pour chaque US, ou groupe d'US, j'ai ensuite réalisé une maquette à main levée sur l'iPad, puis l'ai adaptée sur Figma.

Pour le placement des éléments, je me suis inspirée des sites Blablacar et Ouigo, que j'avais déjà utilisés plusieurs fois, afin de garantir une expérience utilisateur intuitive et familière.

Je vais expliquer ici les maquettes des 3 pages les plus importantes : l'index, proposer un trajet, résultat de recherche.

6.2.1 Maquette “Index”

Mockup web de la page d'accueil

Annexe I.1 : Wireframe web de la page d'accueil

La page d'accueil permet aux utilisateurs de comprendre rapidement l'objectif et la valeur ajoutée du site. La photo présente de jeunes personnes souriantes, symbolisant la convivialité et le renouveau dans le covoiturage. Le slogan “Ensemble, roulons vers un futur plus vert” incite les utilisateurs à adopter le covoiturage comme solution écologique et économique.

J'ai choisi une mise en page claire, hiérarchisée et équilibrée, afin que l'utilisateur identifie immédiatement les informations importantes et les actions possibles.

Le formulaire de recherche permet aux visiteurs de consulter rapidement les trajets proposés sans créer de compte en indiquant la date, la ville de départ, la ville d'arrivée et le nombre de passagers. Ce formulaire a été positionné en haut de la page pour un accès immédiat et intuitif, avec des champs clairement identifiés et des placeholders explicatifs.

La section “Fondateur” présente EcoRide, sa démarche récente et son équipe, permettant de renforcer la crédibilité et la confiance des utilisateurs. La section “Comment ça marche ?” décrit brièvement le fonctionnement du site, visant à convaincre rapidement les utilisateurs de rejoindre EcoRide.

Les témoignages servent à renforcer la confiance en montrant l'expérience positive d'autres utilisateurs. La section “Questions fréquentes” répond aux principales interrogations des utilisateurs, facilitant leur passage à l'action de manière fluide et rassurante.

Le Call to Action (CTA) “Créer un compte” est stratégiquement placé à un moment où l'utilisateur est informé et motivé, pour maximiser les chances d'inscription et d'engagement sur la plateforme.

Mockup mobile de la page d'accueil

Annexe I.2 : Wireframe mobile de la page d'accueil

La version mobile reprend les mêmes sections que la version desktop, adaptées pour une navigation sur smartphone avec une mise en page verticale

Wireframe web de la page d'accueil

Annexe I.3 : Mockup web de la page d'accueil

J'ai ensuite formalisé le wireframe esquissé à main levée sur Figma. À cette étape, j'ai conservé l'organisation générale déjà définie, tout en y intégrant quelques ajustements qui me sont venus durant la conception. J'y ai également introduit des couleurs : du vert pour évoquer la nature, et une touche d'orange pour créer un contraste visuel sans rompre l'harmonie globale. Pour le fond, j'ai opté pour des nuances de gris clair et foncé afin de limiter la fatigue visuelle et maintenir un aspect élégant. Enfin, des arrondis et des espacements ont été ajoutés afin d'assurer une meilleure cohérence visuelle, d'harmoniser les éléments et rendre l'ensemble plus aéré et agréable à parcourir.

Wireframe mobile de la page d'accueil

Annexe I.4 : Mockup mobile de la page d'accueil

6.2.2 Maquette "Proposer un trajet"

Mockup web de la page proposer un trajet

Annexe I.5 : Wireframe web de la page proposer un trajet

La page "Proposer un trajet" permet aux conducteurs inscrits de mettre en ligne une nouvelle offre de covoiturage de manière simple et rapide. J'ai choisi une mise en page claire et structurée afin que l'utilisateur remplit les informations essentielles sans difficulté.

Le formulaire est composé de plusieurs champs obligatoires : la ville de départ, d'arrivée, la date, l'horaire, le nombre de places disponibles. L'objectif est de guider l'utilisateur étape par étape pour éviter toute erreur de saisie.

J'ai positionné le formulaire de sorte à ce que les informations du départ soient d'un côté, et celles de l'arrivée de l'autre côté pour éviter toute confusion. Les indications placées dans le champ (placeholders) permettent de comprendre immédiatement la nature des informations attendues.

Le bouton "Publier le trajet" est placé à la fin du formulaire, bien visible et suffisamment contrasté afin d'inciter l'utilisateur à valider son offre une fois toutes les informations complétées.

Mockup mobile de la page proposer un trajet

Annexe I.6 : Wireframe mobile de la page proposer un trajet

La version mobile reprend les mêmes champs que la version desktop, réorganisés verticalement pour faciliter la saisie sur smartphone.

Wireframe web de la page proposer un trajet

Annexe I.7 : Mockup web de la page proposer un trajet

Le wireframe reprend l'organisation définie dans le mockup, avec les éléments simplifiés et hiérarchisés. Les deux colonnes départ/arrivée sont conservées pour une lecture claire et intuitive.

Wireframe mobile de la page proposer un trajet

Annexe I.8 : Mockup mobile de la page proposer un trajet

La version mobile adapte la mise en page en colonne unique, avec des boutons et champs dimensionnés pour un usage tactile.

6.2.3 Maquette “Résultat de recherche”

Mockup web de la page résultat de recherche

Annexe I.9 : Wireframe web de la page de résultat de recherche

La page “Résultat de recherche” permet aux utilisateurs de visualiser les trajets disponibles correspondant aux critères saisis dans le formulaire de recherche. Les résultats sont présentés sous forme de liste de cartes claires et structurée afin de permettre une comparaison rapide entre les différentes propositions de covoiturage.

Chaque résultat affiche les informations essentielles du trajet : ville de départ, ville d'arrivée, date, nombre de places disponibles et conducteur. L'objectif est de permettre à l'utilisateur d'identifier rapidement le trajet qui correspond le mieux à ses besoins

Mockup mobile de la page résultat de recherche

Cf. Annexe I.10 : Wireframe mobile de la page résultat de recherche

La version mobile présente les cartes de trajets en colonne unique, avec les informations essentielles condensées pour une lecture rapide sur smartphone.

Wireframe web de la page résultat de recherche

Cf. Annexe I.11 : Mockup web de la page résultat de recherche

Le wireframe reprend l'organisation générale en mettant l'accent sur la lisibilité des informations principales et sur la hiérarchisation des résultats afin de faciliter la lecture.

Wireframe mobile de la page résultat de recherche

Cf. Annexe I.12 : Mockup mobile de la page résultat de recherche

Une version mobile a également été réalisée afin d'adapter l'affichage des résultats aux écrans plus petits. Les trajets sont présentés sous forme de blocs empilés verticalement afin d'améliorer la lisibilité et l'ergonomie sur smartphone.

7. Diagrammes de gestion de projet

Dans le cadre de la conception de la plateforme EcoRide, l'outil draw.io a été utilisé afin de créer différents diagrammes permettant de structurer le projet, visualiser les fonctionnalités et organiser les idées avant de se lancer dans le développement.

L'utilisation de cet outil permet de représenter graphiquement le fonctionnement de l'application, les interactions entre les utilisateurs et le système, ainsi que les échanges entre les différentes parties de l'application. Ces diagrammes facilitent la compréhension globale du projet et permettent d'anticiper la logique du site.

Trois types de diagrammes ont été réalisés : un diagramme de cas d'utilisation, un diagramme de séquence et un diagramme de classe.

7.1 *Diagramme de cas d'utilisation*

Cf. Annexe I.13 : Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation permet de représenter les différentes actions que les utilisateurs peuvent effectuer sur la plateforme.

Dans le cadre du projet EcoRide, ce diagramme met en évidence les interactions entre les utilisateurs et le système. Quatre types d'utilisateurs sont représentés :

- Passager
- Conducteur
- Passager-conducteur
- Visiteur

Le diagramme présente les principales fonctionnalités sur la plateforme, telles que :

- Proposer des trajets
- Enregistrer un véhicule

- Participer à des trajets
- Consulter les profils
- Etc.

7.2 *Diagramme de séquence - Connexion utilisateur*

Annexe I.14 : Diagramme de séquence - Connexion utilisateur

Le diagramme de séquence illustre le processus de connexion d'un utilisateur à la plateforme.

Ce diagramme permet de représenter les interactions entre les différents éléments du système :

- le passager
- l'interface utilisateur
- le serveur
- la base de données

Ce diagramme permet de comprendre la circulation des informations entre les différentes parties de l'application à partir du moment où l'utilisateur entre ses informations de connexion jusqu'au moment où la connexion est un succès ou un échec.

7.3 *Diagramme de classe (UML)*

Annexe I.15 : Diagramme de classe (UML)

Le diagramme de classe représente les principales entités de la plateforme EcoRide et leurs relations.

Cinq classes ont été identifiées : utilisateur, véhicule, trajet, réservation et avis. Un utilisateur peut posséder plusieurs véhicules, proposer des trajets ou effectuer des réservations selon son rôle. Chaque réservation est liée à un trajet et peut générer un avis.

Il permet de visualiser clairement les responsabilités de chaque entité et les liens qui les unissent, garantissant une structure cohérente et maintenable pour l'application.

8. Organisation et suivi du projet

8.1 Trello

Annexe I.16 : Screenshot des tâches Trello

L'outil Trello a été utilisé pour organiser et suivre les différentes étapes du projet.

Les différentes tâches du projet ont été organisées sous forme de cartes, chacune correspondant à une fonctionnalité, une amélioration ou une étape du développement. Les cartes ont ensuite été déplacées entre les colonnes "À faire", "En cours", "Fin", ce qui permettait de suivre l'avancement du projet de manière visuelle et structurée.

Cette organisation m'a permis de prioriser les tâches importantes, de garder une vision globale de mes tâches en attente et d'identifier rapidement les points bloquants afin de réajuster ma planification.

8.2 Git

Pour la gestion du code source et le suivi des modifications, les outils Git et GitHub ont été utilisés tout au long du projet.

L'utilisation de Git permet également de créer des branches. Celles-ci offrent la possibilité de développer de nouvelles fonctionnalités ou d'effectuer des tests sans modifier directement la version principale du projet. Une fois les modifications validées, elles peuvent être fusionnées avec la branche principale.

L'apprentissage de ces outils a demandé un certain temps d'adaptation afin de bien comprendre leur fonctionnement et les différentes commandes associées. Avec la pratique, leur utilisation est devenue plus intuitive et ils se sont révélés essentiels pour organiser le développement et sécuriser les différentes versions du projet.

8.3 To-do lists

En complément de Trello, des to-do lists quotidiennes ont été utilisées afin d'organiser le travail au jour le jour en tâches plus petites et de maintenir un rythme de progression régulier.

Cette méthode, bien que simple d'utilisation, m'aide à rester concentrée sur les priorités et à éviter l'accumulation du travail.

Screenshot des to-do lists :

Bloc 1 (Jour 1) – 3h ☐

- Lister tous les bugs HTML/CSS ☐
 - Classer bugs : critiques vs esthétiques ☐
 - Corriger les 50% des bugs critiques (titres, images, sections principales) ☐
-

Bloc 2 (Jour 2) – 3h ☐

- Corriger le reste des bugs critiques et bugs secondaires ☐
- Vérifier compatibilité sur différents navigateurs ☐
- Test complet front-end ☐

II. Développement

Le développement a été réalisé sur Visual Studio Code 2, avec une organisation claire des fichiers et un code commenté tout au long du projet afin de faciliter la lecture, la compréhension et la maintenance.

Annexe I.17 : screenshot code commenté

1. Front-end

Le front-end du projet EcoRide repose sur les technologies suivantes :

- HTML5
- CSS3 : inclus dans le <head> de chaque page via la balise <link rel="stylesheet">
- JavaScript ES6 : inclus avant le </body> de chaque page via <script src="...">

1.1 *HTML - Structure des pages*

1.1.1 *Organisation générale*

Cette partie décrit la structure HTML commune à toutes les pages du site EcoRide.

- **Header commun**
 - Logo et nom du site avec lien vers la page d'accueil
 - Menu de navigation principal : Accueil, Proposer un trajet, Rechercher un covoiturage.
 - Profil utilisateur avec redirection vers le profil si connecté, ou vers la page de connexion/inscription si non connecté (géré en PHP)

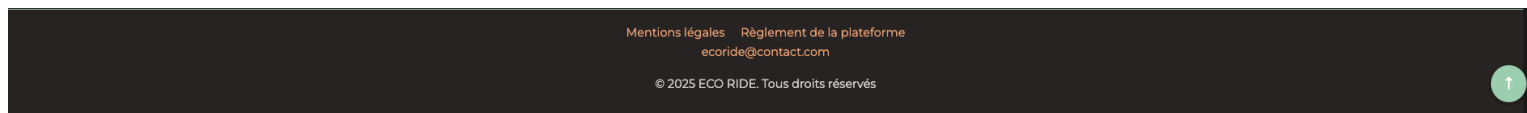
Screenshot du header commun :



- **Footer commun**

- Liens légaux : Mentions légales, Règlement, Contact.
- Bouton de déconnexion si l'utilisateur est connecté.
- Copyright et informations EcoRide.

Screenshot du footer commun :



- **Structure générale du HTML :**

Chaque page suit une structure HTML standard : `<!DOCTYPE html>`, `<html lang='fr'>`, un `<head>` incluant les métadonnées et les feuilles de styles, et un `<body>` structuré avec le header et le footer communs inclus via PHP, les sections spécifiques à chaque page, et les scripts JS en fin de page.

Cette organisation garantit une cohérence visuelle et fonctionnelle sur toutes les pages et facilite la maintenance et les évolutions futures.

1.1.2 Pages principales

1.1.2.1 Page "Index"

Cette partie détaille le contenu spécifique à la page d'accueil. Chaque section est pensée pour l'expérience utilisateur et la conversion.

- *Section Hero / Accueil*

- Image de fond (libre de droit) représentant la convivialité et le covoiturage.
- Slogan principal : “Ensemble, roulons vers un futur plus vert”.

Objectif : capter immédiatement l’attention et expliquer la valeur ajoutée du site.

Screenshot du hero de la page d’accueil :



- *Formulaire de recherche rapide*

- Champs : Ville de départ, Ville d’arrivée, Date, Nombre de passagers.
- Positionné en haut pour un accès immédiat.
- Placeholders explicatifs et labels pour faciliter la compréhension.
- Bouton de recherche avec icône.

Objectif : permettre aux utilisateurs de trouver rapidement un trajet, même sans compte.

Screenshot du formulaire de recherche de la page d’accueil :

- *Section fondateur*
 - Photo et biographie de José Marceau, fondateur.
 - Description de la vision et de l'ambition d'EcoRide.

Objectif : renforcer la crédibilité et la confiance des utilisateurs.

Screenshot de la section fondateur :



José Marceau

Originaire d'Annecy, José Marceau a fondé Eco Ride en 2022 avec une conviction forte : rendre les modes de transport durables plus visibles, accessibles et attractifs. Après plusieurs années à travailler dans le secteur associatif et environnemental, il constate que de nombreuses initiatives locales peinent à se faire connaître, malgré leur impact positif. C'est ainsi qu'est née Eco Ride : une plateforme dédiée aux mobilités douces, au service de celles et ceux qui souhaitent se déplacer autrement, à leur échelle. José croit en un changement progressif, porté par l'information, la confiance et des solutions concrètes. À travers Eco Ride, il souhaite créer un lien entre les citoyens, les acteurs locaux et les alternatives de transport, dans un esprit d'ouverture, de simplicité et de respect de l'environnement.

- *Section "Comment ça marche?"*
 - Étapes numérotées pour expliquer le fonctionnement : recherche pour les passagers, et proposition pour les conducteurs

Objectif : guider les nouveaux utilisateurs et réduire les frictions à l'inscription.

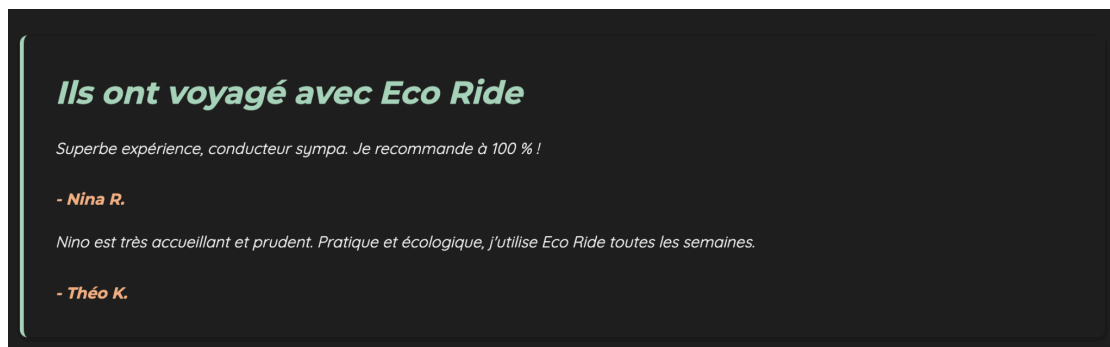
Screenshot de la section “Comment ça marche ?” :



- Témoignages
 - Retours positifs d'utilisateurs

Objectif : renforcer la confiance et la preuve sociale.

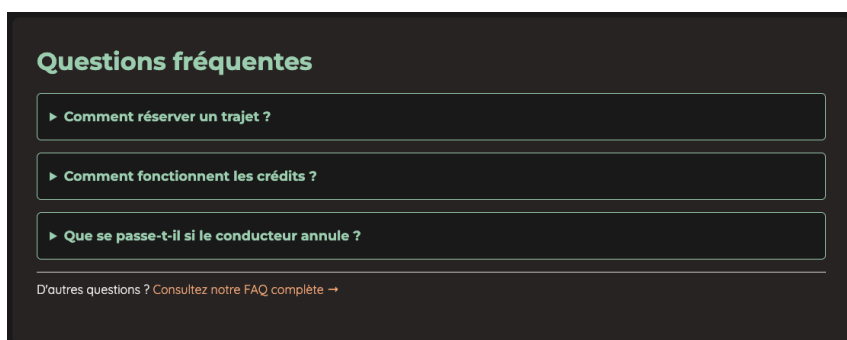
Screenshot de la section témoignages :



- FAQ
 - Questions et réponses principales (ex : créer un compte, réserver un trajet)

Objectif : anticiper les interrogations et faciliter le passage à l'action.

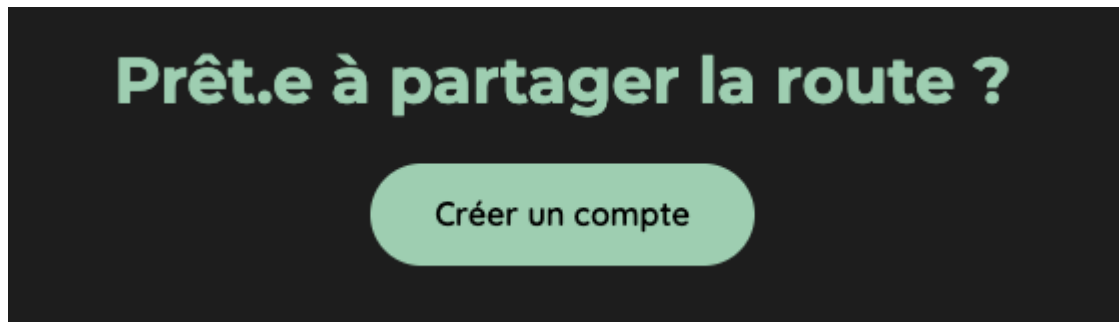
Screenshot de la section FAQ :



- *Call to Action (CTA)*
 - Bouton “créer un compte” bien visible et contrasté
 - Position stratégique après avoir présenté l’offre et les témoignages

Objectif : maximiser l’inscription et l’engagement

Screenshot du CTA final :



1.1.2.2 *Page “Proposer un trajet”*

Annexe II.1 : Screenshot de la page “Proposer un trajet”

- *Titre et instructions :*
 - Titre “Votre trajet - Étape 1 sur 2”.
 - Note indiquant que certains champs sont obligatoires.

Objectif : clarifier la progression et éviter les erreurs lors de la saisie.

- *Formulaire de proposition de trajet*
 - Colonne de gauche - Informations de départ :
 - Adresse de départ
 - Arrêts optionnels, avec possibilité d’ajouter dynamiquement des arrêts via JS
 - Sélection du véhicule utilisé parmi les véhicules de l’utilisateur
 - Date et heure de départ

Objectif : guider l'utilisateur pour qu'il fournisse toutes les informations nécessaires sans confusion.

- Colonne de droite - Informations d'arrivée :
 - Adresse d'arrivée
 - Nombre de places disponibles
 - Autres précisions (optionnel) : passage par autoroute, bagages, ...
 - Affichage des crédits gagnés par passager.
 - Boutons de validation

Objectif : faciliter la lecture et la saisie des informations essentielles pour le trajet.

1.1.2.3 *Page "Résultat de recherche"*

Annexe II.2 : screenshot de la page "Résultat de recherche"

- Formulaire de recherche (en haut de la page)
 - Formulaire avec champs pré-remplis avec les valeurs saisies par l'utilisateur, s'il vient de la page Index ; ou champs vides, si l'utilisateur est arrivé sur cette page via le menu

Objectif : permettre à l'utilisateur de modifier facilement ses critères et de lancer/relancer la recherche sans revenir à la page d'accueil.

- Section des filtres
 - Note minimale du chauffeur, prix maximum, aspect écologique et durée du voyage
 - Bouton "tout effacer" pour réinitialiser les filtres

Objectif : améliorer l'expérience utilisateur en permettant un tri rapide des résultats.

- Résultats de recherche
 - Liste des trajets correspondant aux critères

- Pour chaque trajet :
 - Informations sur le chauffeur (photo, note et nom)
 - Départ, arrivée, heure et date, nombre de places disponibles
 - Bouton “voir détails” redirigeant vers la page des détails du trajet
 - Badge écologique si le véhicule est électrique
- Si aucun trajet trouvé, proposition de la date du prochain trajet disponible

Objectif : présenter de manière claire et hiérarchisée toutes les informations importantes pour le choix d’un trajet.

1.1.3 *Pages secondaires*

Les pages secondaires du site reprennent la structure HTML commune à toutes les pages, avec le header, le footer et une organisation claire en sections pour assurer une navigation uniforme et cohérente. Certaines pages sont centrées sur le profil utilisateur, comme la page “Informations personnelles” ou la page de visualisation des profils des autres utilisateurs. D’autres pages, comme la page “détails du trajet”, présentent des cartes ou blocs d’informations pour le conducteur, l’itinéraire et les passagers, avec des titres et paragraphes hiérarchisés (<h2>, <h3>, <p>) pour faciliter la lecture.

Toutes les pages secondaires respectent le principe de lisibilité et de hiérarchie, afin que l’utilisateur identifie rapidement les informations importantes et les actions possibles.

En plus des pages accessibles aux utilisateurs, le site comprend également des pages destinées aux employés d’EcoRide, comprenant les rôles d’administrateur et d’employé standard. Ces pages HTML utilisent des tableaux <table> pour l’affichage des comptes, des articles <article> et des blocs <div> pour les signalements, ainsi que des boutons <button> et formulaires <form> pour les actions. Elles respectent le même principe de structure HTML que les pages utilisateurs, garantissant une présentation claire et hiérarchisée des informations.

1.2 *CSS - Design, styles et responsive*

Design et styles

Pour le choix des couleurs, j'ai opté pour un fond sombre afin de réduire la consommation énergétique sur les écrans compatibles (OLED), complété par deux couleurs principales, orange et vert, apportant modernité et rappelant la nature.

Le logo a été créé par mes soins sur *Procreate*, en utilisant les couleurs vert et orange, et a évolué de la version 1 à la version 2, par souci de lisibilité compte tenu de la petite taille du logo.

Logo version 1 :



Logo version 2 :



Pour l'organisation des fichiers CSS, j'ai créé un petit fichier CSS commun pour tout ce qui est couleur et police, puis j'ai créé un fichier CSS par page pour les styles plus précis, afin de limiter les responsabilités et faciliter la maintenance. Les éléments réutilisables comme

les headers, footers et menus ont été développés en composants PHP, inclus via include, afin de simplifier le code et éviter les répétitions.

Les boutons, tableaux et formulaires utilisent des styles définis dans le CSS commun, avec bordures arrondies, ombres portées, couleurs cohérentes avec la charte, et transitions pour le survol. Les tableaux affichent leurs éléments avec un contraste clair (par ex. en-tête orange, contenu vert doux sur fond beige foncé). Des effets de profondeur sont créés grâce aux différentes nuances de chaque couleur et aux ombres portées.

Pour l'index, la section .hero intègre une image en arrière-plan avec un filtre brightness pour lisibilité, et un slogan central en overlay avec fond semi-transparent et arrondi.

La typographie du site est uniforme, utilisant Montserrat et Quicksand via Google Font. Les titres suivent une hiérarchie claire avec des tailles et graisses différentes, afin de mieux les distinguer.

L'ensemble du CSS suit une logique claire :

- Variables CSS pour les couleurs (:root) pour faciliter les modifications globales :

```
:root {  
  --vert-doux: #a8d5ba ;  
  --orange-doux: #f4b183 ;  
  --beige-fonce: #2e2b28 ;  
  --gris-doux: #e0e0e0 ;  
  --texte: #f0f0f0 ;  
  --fond: #1f1f1f ;  
  --shadow: rgba(0, 0, 0, 0.3)  
}
```

- Titres hiérarchisés et paragraphes espacés pour la lisibilité.

- Boutons, tableaux et formulaires cohérents avec la charte graphique :
- Transitions douces pour les interactions (hover, focus) :



Responsive

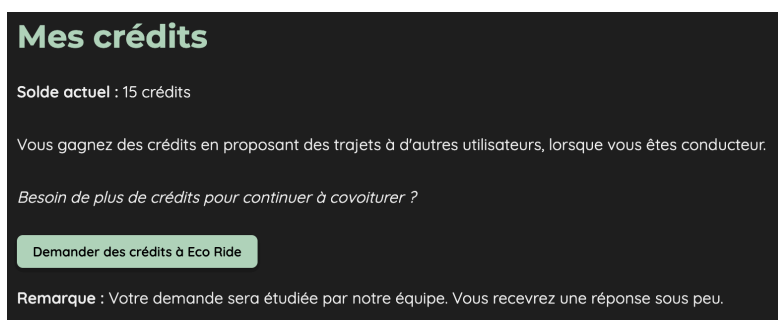
Annexe II.3: exemple d'éléments version desktop et version mobile

L'interface a été pensée pour être responsive, c'est-à-dire s'adapter à la fois aux écrans d'ordinateur, tablette et mobile.

Le site a été développé en desktop first, ce qui signifie que les styles principaux sont conçus pour l'affichage sur ordinateur, puis adaptés aux écrans plus petits grâce aux techniques de responsive design.

Les techniques utilisées incluent :

- Flexbox, pour organiser et aligner les conteneurs et composants de manière flexible selon la taille de l'écran.
- Media queries, pour ajuster la taille, la disposition et les composants selon la largeur de l'écran.
- Unités relatives (% , em, rem) afin que les éléments s'adaptent proportionnellement aux différentes résolutions.



Exemples d'adaptations :

- Les searchbars s'adaptent en colonne sur mobile, avec les champs et boutons en pleine largeur.
- Les éléments se repositionnent ou deviennent scrollables selon la largeur de l'écran grâce aux media queries.
- Les titres, boutons, tableaux, formulaires et sections s'adaptent pour rester lisibles et cohérents sur tous les supports.
- Flexbox et responsive sont appliqués sur les conteneurs et composants.
- Des points précis de breakpoints sont utilisés, par exemple 768px pour tablette et 480px pour smartphone.
- Certains éléments changent selon la taille de l'écran : flex-direction, tailles de police, largeur des champs et boutons.

1.3 *Javascript - Interactions*

JavaScript est utilisé afin d'améliorer l'expérience utilisateur sans rechargement complet de la page. Il permet notamment d'ajouter des comportements dynamiques à l'interface, d'améliorer la navigation et de rendre certaines interactions plus fluides pour l'utilisateur. Il permet également de manipuler dynamiquement le DOM (Document Object Model), c'est-à-dire la structure HTML de la page, afin de modifier ou afficher certains éléments sans recharger l'ensemble du contenu.

Les fonctionnalités JavaScript mises en place comprennent :

- ouverture et fermeture des modales
- affichage de messages de confirmation (toasts)
- gestion dynamique de l'interface utilisateur

Les toasts permettent d'informer l'utilisateur du succès d'une action (modification d'informations, publication d'un trajet). Ils apparaissent temporairement puis disparaissent automatiquement après quelques secondes afin de ne pas gêner la navigation.

Toast de succès :

✓ Informations mises à jour avec succès.

Les modales sont utilisées pour afficher certaines actions ou formulaires sans quitter la page courante. L'utilisateur peut les fermer via un bouton dédié ou en cliquant en dehors de la fenêtre modale.

Modal pour modification des informations :

The image shows a modal window titled "Modifier mes informations" with a close button (X) in the top right corner. The modal contains several form fields and labels:

- Photo de profil :** A file upload area with a button "Choisir un fichier" and the text "Aucun fichier choisi".
- Prénom :** A text input field containing "Clara". Below it, a small red message says "Merci de contacter Eco Ride pour modifier cette information".
- Nom :** A text input field containing "Soual". Below it, a small red message says "Merci de contacter Eco Ride pour modifier cette information".
- Email :** A text input field containing "clara.soual@example.com". Below it, a small red message says "Merci de contacter Eco Ride pour modifier cette information".
- Date de naissance :** A label with a corresponding input field partially visible at the bottom.

At the bottom of the modal, there are two radio buttons: "Passager" (selected) and "Conducteur".

Fetch API - Communications asynchrones

Le projet utilise l'API Fetch native de JavaScript (ES6) pour effectuer des communications asynchrones avec le serveur sans rechargement de page. Fetch a été préféré à Ajax pour sa syntaxe plus moderne et lisible, et à Axios car il est natif au navigateur et ne nécessite aucune dépendance externe.

Fetch est utilisé dans le projet pour la gestion des avis employé : lorsqu'un employé clique sur "Valider" ou "Refuser" un avis, une requête Fetch est envoyée vers `traiter-avis.php` qui met à jour le statut en base de données et renvoie une réponse JSON. La

ligne de l'avis disparaît alors du tableau sans recharger la page, et un toast de confirmation s'affiche.

Exemple de requête Fetch utilisée :

```
fetch('../PHP/traiter-avis.php', {  
  method: 'POST',  
  body: formData  
})  
.then(res => res.json())  
.then(data => {  
  if (data.success) {  
    ligne.remove();  
    afficherToast('✅ Avis validé !');  
  }  
});
```

Fonctionnalités JavaScript plus spécifiques :

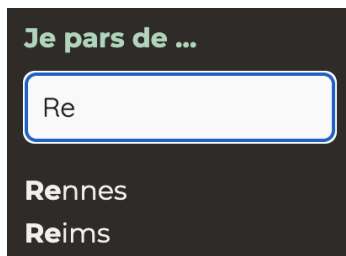
Certaines pages du site possèdent également des scripts JavaScript dédiés afin d'améliorer leur ergonomie et leur interactivité.

Page Index

Sur la page d'accueil, plusieurs interactions ont été ajoutées :

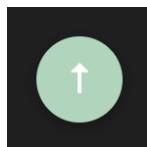
- bouton "remonter en haut" avec une flèche apparaissant automatiquement lors du scroll

- vérification simple des champs du formulaire de recherche avant soumission (format mail, nombre max de passagers, etc.)
- système d'autocomplétion pour les villes
- affichage dynamique de suggestions de villes lors de la saisie



Ces fonctionnalités reposent notamment sur l'écoute d'événements utilisateurs (event listeners) et sur la manipulation dynamique du DOM afin d'afficher ou masquer certains éléments en fonction des actions de l'utilisateur.

L'autocomplétion permet d'améliorer la rapidité de saisie et de limiter les erreurs de frappe dans les champs de recherche.



Page "Proposer un trajet"

Annexe II.4 : Screenshot de l'animation JavaScript

Après la publication d'un trajet, une animation de confettis est générée afin de marquer visuellement la réussite de l'action et améliorer l'expérience utilisateur.

Ces interactions JavaScript permettent d'améliorer l'ergonomie générale du site en rendant l'interface plus dynamique, réactive et intuitive pour l'utilisateur.

2. Back-end

Le back-end du projet EcoRide repose sur les technologies suivantes :

- PHP 8 : inclus dans les pages via `require_once` ou `include`
- MySQL 8 : connecté via PDO dans `connexion.php`
- JSON : logs écrits et lus via les fonctions du fichier `logs.php`

2.1 *PHP et Logique métier*

La structure du projet a été mise en place afin de respecter une séparation claire des responsabilités entre l’affichage, la logique métier et les ressources.

Cette organisation repose sur une séparation en trois couches :

- la couche présentation (affichage des pages via HTML/PHP),
- la couche traitement (logique métier et traitement des formulaires)
- la couche accès aux données (connexion et requêtes vers la base de données via PDO).

Cette structuration permet d’éviter le mélange des responsabilités dans un même fichier, facilite la maintenance du projet et la réutilisation du code, et assure une organisation claire et modulable.

Les fonctionnalités principales développées en PHP incluent :

- Authentification et gestion des sessions
- Validation côté serveur : vérification des champs obligatoires, validation des formats (email, date, nombre de passagers), nettoyage des entrées utilisateurs (`trim()`), conversions, valeurs par défaut)
- Gestion des données : insertion et mises à jour dans la base via requêtes préparées PDO, prévention des injections SQL
- Redirections dynamiques : redirection vers la page précédente après connexion grâce au paramètre `redirect`
- Protection des accès directs : vérification de la méthode HTTP (POST) avant de traiter un formulaire ou un script

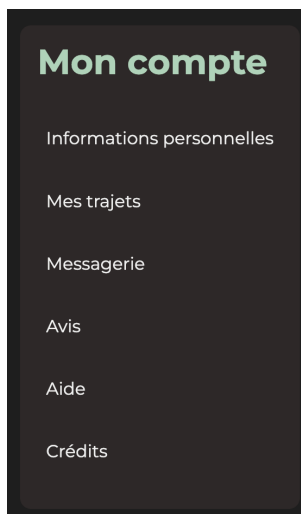
Cette approche permet d'isoler la logique réutilisable dans des fonctions, d'améliorer la lisibilité et de sécuriser les opérations sensibles.

2.2 Composants réutilisables

Les composants réutilisables sont gérés via PHP (include/require) pour intégrer les mêmes blocs dans plusieurs pages sans duplication et pour faciliter leur maintenance. Cela permet également de garantir une cohérence graphique et fonctionnelle sur tout le site.

Parmi les composants utilisés :

- Header
- Footer
- Menu de profil



- Menu de la partie employés
- Menu de la partie administrateur

En complément, certaines fonctions PHP réutilisables encapsulent de la logique métier commune, ce qui évite de répéter du code et renforce la lisibilité du projet :

- Gestion des sessions et authentification : `loginUser()`, `logoutUser()`, `requireLogin()`

- Vérification de l'accès aux pages sensibles selon l'état de connexion ou le rôle de l'utilisateur

L'utilisation de composants et fonctions réutilisables permet d'optimiser le développement, de maintenir la cohérence du site et de centraliser la gestion de la logique commune, tout en facilitant la maintenance et l'évolution future du projet.

2.3 *Sécurité et validation*

Chaque fichier est développé en respectant les bonnes pratiques :

- utilisation des requêtes préparées avec PDO afin de prévenir les injections SQL
- gestion des erreurs avec PDO::ERRMODE_EXCEPTION pour faciliter le débogage et le traitement des erreurs
- échappement des données affichées avec htmlspecialchars() afin de prévenir les attaques XSS (Cross-Site Scripting)
- validation des données côté serveur

Les champs de formulaire sont systématiquement contrôlés côté serveur avant tout traitement :

- vérification des champs obligatoires
- validation du format des emails
- vérification de l'unicité de l'email
- contrôle de la méthode HTTP utilisée (POST) avant le traitement des formulaires

Double validation JS + PHP :

Tous les formulaires du projet sont soumis à une double validation : une première validation côté client en JavaScript, et une seconde côté serveur en PHP.

Côté client (JavaScript) : avant l'envoi du formulaire, JS vérifie que les champs obligatoires sont remplis, que les formats sont corrects (email, date, nombre) et affiche des messages d'erreur directement sous chaque champ concerné. L'attribut novalidate est

ajouté sur chaque formulaire afin de désactiver la validation native du navigateur et laisser le JS gérer entièrement cette étape.

Côté serveur (PHP) : même si la validation JS est passée, les données sont revérifiées à réception. Cette double vérification est indispensable car le JS peut être désactivé ou contourné par un utilisateur malveillant. Le PHP utilise des expressions régulières (`preg_match()`), des conversions de type (`(int)`, `(float)`), et `htmlspecialchars()` pour nettoyer et sécuriser toutes les entrées avant tout traitement ou enregistrement en base de données.

Authentification et sécurité des utilisateurs :

- mots de passe hashés par `password_hash()` utilisant l'algorithme bcrypt
- vérification des mots de passe avec `password_verify()` lors de la connexion
- vérification des identifiants via la BDD et création de sessions PHP
- communication sécurisée prévue via HTTPS

Gestion des sessions PHP

Lors de la connexion, PHP démarre une session avec `session_start()` et enregistre l'identifiant de l'utilisateur dans `$_SESSION['user_id']`. Cette variable est ensuite vérifiée sur chaque page protégée via la fonction `requireLogin()`, qui redirige automatiquement vers la page de connexion si l'utilisateur n'est pas authentifié. À la déconnexion, la session est détruite avec `session_destroy()`. Ce mécanisme permet de maintenir l'état de connexion entre les différentes pages du site sans avoir à re-authentifier l'utilisateur à chaque requête.

PHP crée automatiquement un cookie `PHPSESSID` dans le navigateur pour maintenir l'identifiant de session, les données sensibles restant stockées côté serveur.

Concernant l'authentification par token, JWT (JSON Web Token) est une alternative aux sessions adaptée aux API REST. EcoRide reposant sur une architecture PHP classique avec génération de pages côté serveur, les sessions PHP sont plus appropriées que JWT dans ce contexte.

En complément, certaines pages vérifient que l'utilisateur est connecté avant d'effectuer des actions sensibles (modification de profil, réservation et publication de trajet).

Un contrôle d'accès est également réalisé afin de restreindre certaines fonctionnalités selon le rôle de l'utilisateur (administrateur, employé, conducteur, passager, visiteur).

2.4 *Dashboard admin*

Annexe II.5 : pages de la section admin

L'espace administrateur est une interface sécurisée, accessible uniquement après authentification via une page de connexion dédiée (ADM-login.php). Comme demandé, la création des comptes administrateur s'est faite directement en base de données.

L'identifiant de session est stocké sous la clé \$_SESSION['admin_id'], distincte de celle des utilisateurs classiques, afin d'isoler les accès. La déconnexion détruit la session via session_destroy().

Le dashboard se compose de quatre espaces : la gestion des employés (créer, modifier, suspendre/réactiver via popups JS), la gestion des utilisateurs (liste avec recherche dynamique côté client, profil détaillé, suspension), les statistiques (deux graphiques [Charts.js](#) sur période personnalisable : covoiturages par jour depuis MySQL, crédits gagnés depuis transactions.json), et la page des logs (historique JSON des actions, filtres par niveau et action, pagination, vidage des logs).

2.5 *Dashboard employés*

Annexe II.6 : pages de la partie employé

L'espace employé est sécurisé via une page de connexion dédiée (EMP-login.php). La session est stockée sous la clé \$_SESSION['employee_id'], distincte de celle des administrateurs et utilisateurs. Chaque page appelle requireEmploye() définie dans auth.php pour vérifier la session avant tout affichage.

Le dashboard se compose de trois espaces. La modération des avis (EMP-gestion-avis.php) affiche les avis filtrés par statut et permet de les valider, refuser,

remettre en attente ou supprimer via Fetch API sans rechargement, avec retour visuel par toast. La gestion des litiges (EMP-litiges.php) liste les covoiturages signalés (passager→conducteur et conducteur→passager), une modale détaille les contacts, le motif et permet à l'employé d'ajouter une note interne, puis de débloquer, bloquer les crédits d'un conducteur, le suspendre, ou suspendre un passager. Les demandes de crédits (EMP-demande-credits.php) traitent les demandes stockées en JSON : l'acceptation crédite automatiquement l'utilisateur en base de données et enregistre la transaction dans transactions.json.

3. Fonctionnalités détaillées

Fonctionnalité : proposer un covoiturage

Objectif de la fonctionnalité

Cette fonctionnalité permet à un utilisateur ayant le rôle de conducteur ou passager-conducteur de créer un nouveau trajet.

Elle est essentielle car elle permet d'alimenter la base de données en trajets disponibles, et d'alimenter le site en contenu réel et le faire fonctionner.

Description fonctionnelle

Annexe II.7 : page proposer un trajet

L'utilisateur accède à une page contenant un formulaire permettant de saisir les informations du trajet qu'il veut proposer, tels que : adresses de départ et d'arrivée, date et heure, nombre de places disponibles, etc.

Annexe II.8 : page proposer trajet 2

Après validation du formulaire, les données sont envoyées vers la page proposer un trajet 2, qui récapitule les informations saisies à l'étape précédente.

Après avoir vérifié, il peut soit appuyer sur modifier, pour retourner sur la page précédente, soit valider.

Annexe II.9 : page de confirmation

Une fois le récapitulatif validé, les données sont envoyées au serveur, vérifiées, le trajet est enregistré dans la base de données et l'utilisateur est redirigé vers la page de succès proposer un trajet 3.

Logique technique

Cette fonctionnalité repose sur :

- HTML → affichage du formulaire
- PHP → traitement des données
- SQL → enregistrement du trajet
- JavaScript → ajout dynamique des arrêts, mémorisation temporaire des données via sessionStorage

Extraits de code

Extrait 1 - Formulaire HTML

Annexe II.10 : extrait de code du formulaire sur la page proposer trajet

Ce formulaire permet à l'utilisateur de saisir les informations du trajet.

L'attribut required empêche l'envoi du formulaire si les champs obligatoires ne sont pas remplis.

Extrait 2 - Récupération des données PHP

Cf. Annexe II.11 : extrait de code de récupération des données en PHP

Les données envoyées par le formulaire sont récupérées grâce au tableau \$_POST.
La fonction trim() permet de supprimer les espaces inutiles.
Cela évite d'enregistrer des valeurs incorrectes dans la base de données.

Extrait 3 - Récupération des arrêts dynamiques (JavaScript)

Annexe II.12: extrait de code du JS pour la récupération des arrêts

Ce code permet de récupérer automatiquement tous les arrêts ajoutés dynamiquement par l'utilisateur.
Les champs nommés "step 1", "step 2", etc. sont détectés puis ajoutés dans un tableau.

Extrait 4 - Insertion dans la base de données

Annexe II.13 : extrait du code php pour l'insertion SQL

Cette requête permet d'enregistrer le trajet dans la base de données.
L'utilisation de prepare() permet de sécuriser la requête contre les injections SQL.

Extrait 5 - Redirections

De l'étape 1 à 2 :

La redirection se fait côté serveur, dans le PHP de la page. Après que l'utilisateur ait rempli le formulaire et validé, les données sont récupérées et enregistrées en session.

Le code PHP utilise :

```
header('Location: proposer_trajet-2.php');
```

```
exit;
```

pour envoyer automatiquement vers l'étape 2, qui affiche un résumé de trajet avant confirmation.

De l'étape 2 à 3 :

Cette redirection se fait côté client, via un formulaire HTML dans la page de l'étape 2 :

```
<form action="USR-proposer-trajet-3.php" method="POST">  
  
<button type="submit" class="btn-submit">Confirmer le trajet</button>  
  
</form>
```

Quand l'utilisateur clique sur "Confirmer le trajet", le formulaire est soumis, l'utilisateur est redirigé vers la dernière page de confirmation, et le trajet est enregistré dans la BDD.

Extrait 6 - Ajout dynamique des arrêts (JavaScript)

Annexe II.14 : extrait de code pour l'ajout dynamique des arrêts

Ce script JavaScript permet à l'utilisateur d'ajouter dynamiquement des arrêts intermédiaires dans le trajet.

Lorsque l'utilisateur clique sur le bouton, un nouveau champ est automatiquement ajouté dans le formulaire.

La variable `existingStops.length` permet de compter le nombre d'arrêts existants afin d'attribuer automatiquement un numéro au nouvel arrêt.

Une condition limite également le nombre d'arrêts : `if(stopNumber > 6) →` pour éviter un formulaire trop long.

Fonctionnalité : voir mes trajets

Objectif de la fonctionnalité

La fonctionnalité mes trajets permet à un utilisateur connecté de visualiser tous les trajets auxquels il est lié, que ce soit en tant que passager ou conducteur.

Elle est essentielle car elle permet à l'utilisateur d'avoir un tableau de bord clair de tous ses trajets et d'accéder facilement aux actions disponibles selon son rôle.

Description fonctionnelle

Annexe II.15 : page mes trajets

L'utilisateur accède à une page contenant trois sections : trajets à venir, trajets en cours, trajets passés.

Chaque section liste les trajets correspondants. Pour chaque trajet, l'utilisateur voit toutes les informations essentielles tels que son rôle, le conducteur ou le.s passager.s, la ville de départ et d'arrivée, l'heure, etc. ainsi qu'un bouton voir détails.

Si aucun trajet n'existe pour cette section, un message informatif est affiché.

Page détails du trajet

Annexe II.16 : page détail du trajet

Dans le cas actuel, l'utilisateur arrive sur la page détail du trajet après avoir cliqué sur "voir détails" dans la section "mes trajets".

La page affiche l'intégralité des informations du trajet, avec notamment les passagers déjà inscrits, avec des liens vers leurs profils.

Les actions possibles changent selon le rôle que l'utilisateur a dans ce trajet : s'il est conducteur, il peut modifier le trajet ou le supprimer ; s'il est passager inscrit : il peut se désinscrire ; s'il n'est pas impliqué, il peut réserver une place.

Lorsqu'un utilisateur veut voir les détails d'un trajet passé, un message l'informant que le trajet est terminé s'affiche.

Logique technique

Récupération et tri des trajets :

PHP récupère depuis la base de données les trajets où l'utilisateur est conducteur et/ou passager. Ensuite, les trajets sont triés par date et par heure pour être placés correctement dans les 3 sections.

Différenciation des rôles :

PHP vérifie le rôle de l'utilisateur pour chaque trajet afin d'afficher les boutons et actions appropriées : la page détails utilise les variables \$isOwner et \$isPassenger pour gérer l'affichage conditionnel des boutons et modals.

Extraits de code

Extrait 1 - Récupération des trajets en PHP

Annexe II.17 : extrait de code récupération des trajets en PHP

Ce code récupère tous les trajets liés à l'utilisateur, qu'il soit conducteur ou passager, et fusionne les résultats pour un affichage global.

Extrait 2 - Tri des trajets selon la date

Annexe II.18 : extrait du code tri des trajets selon la date en PHP

Cette logique classe les trajets dans les sections "À venir", "En cours", et "Passés" pour faciliter la lecture et la navigation.

Extrait 3 - Affichage d'un trajet

Annexe II.19 : code PHP pour l'affichage d'un trajet

Cette fonction centralise l'affichage de chaque trajet avec les informations essentielles et le lien vers la page de détails, en masquant les informations sensibles ou non pertinentes.

Extrait 4 - Page détails et rôle de l'utilisateur

Annexe II.20 : code PHP de vérification du propriétaire

Ces variables permettent de savoir si l'utilisateur peut modifier le trajet, se désinscrire, ou réserver. Elles conditionnent l'affichage des boutons et modals.

Extrait 5 - Gestion du modal modification (conducteur)

Annexe II.21 : code du modal de modification de trajet

Si le trajet n'est pas passé, le conducteur peut ouvrir le modal pour modifier l'élément qu'il souhaite, le formulaire est le même que celui de la page "proposer un trajet".

4. Git et versioning

Un dépôt Git a été mis en place afin de gérer les versions du projet et suivre l'évolution du code tout au long du développement.

Le dépôt distant a été créé sur GitHub afin de sauvegarder le projet et permettre la synchronisation entre l'environnement local et le dépôt Git.

Toutes les commandes citées ci-dessous sont exécutées via le terminal.

Workflow de la création du dépôt :

Création du dépôt local puis initialisation du dépôt Git dans le dossier du projet :

- **cd /Applications/XAMPP/xamppfiles/htdocs/eco_ride** : Se déplacer dans le dossier du projet avant d'exécuter les commandes Git
- **git init** : Initialiser un nouveau dépôt Git dans le dossier du projet

Création du dépôt distant sur GitHub puis liaison entre le dépôt local et le dépôt distant :

- **git remote add origin** : associer le dépôt local au dépôt distant GitHub.
- **git add .** : Ajouter tous les fichiers modifiés au suivi de git pour préparer le commit

- **git commit -m "message"** : valider les changements avec un message expliquant ce qui a été modifié, (ex: "création du formulaire", "validation des champs côté serveur", etc.)
- **git push origin main** : envoyer les commits locaux vers le dépôt distant (main)

Workflow de développement :

Afin de travailler de manière organisée et éviter les conflits dans le code, le développement des nouvelles fonctionnalités est réalisé via un système de branches.

- **git branch nouvelle_fonctionnalité** : créer une nouvelle branche pour travailler sur une fonctionnalité sans toucher à la branche principale
- **git checkout nouvelle_fonctionnalité** : se déplacer sur la branche créée pour commencer à coder

Les modifications sont ensuite réalisées sur cette branche, puis enregistrées avec des commits réguliers afin de garder un historique clair du développement avec :

- **git add .**
- **git commit -m "message"**

Une fois la nouvelle fonctionnalité terminée et testée localement, la branche est envoyée vers GitHub :

- **git push origin nouvelle_fonctionnalité**

Après vérification du bon fonctionnement de la fonctionnalité, les modifications sont fusionnées dans la branche principale.

- **git checkout main** : revenir sur la branche principale
- **git merge nouvelle_fonctionnalité** : fusionner les changements de la branche de développement dans la branche principale après validation

- **git branch -d nouvelle_fonctionnalité** : supprimer la branche locale devenue inutile après la fusion

Commandes de suivi de dépôt :

- **git status** : vérifier l'état des fichiers (modifiés, suivis, non suivis) dans le dépôt
- **git log** : afficher l'historique des commits pour voir les changements réalisés

Authentification sécurisée avec GitHub

Pour sécuriser l'authentification entre mon environnement local et le dépôt GitHub, une clé SSH a été générée sur mon Mac puis ajoutée à mon compte GitHub.

Cela permet d'établir une connexion sécurisée entre la machine locale et GitHub sans avoir à utiliser de token ou de mot de passe à chaque push.

La connexion a ensuite été testée via le terminal afin de vérifier que l'authentification SSH fonctionnait correctement.

III. Base de données

1. Modélisation relationnelle

Annexe III.1 : Diagramme de la base de données EcoRide.

La base de données relationnelle repose sur MySQL et a été conçue afin de stocker de manière sécurisée les informations des utilisateurs et des trajets.

Elle suit une modélisation normalisée permettant d'assurer l'intégrité des relations entre les différentes informations et de faciliter la maintenance du projet.

Plusieurs scripts SQL de création des tables ont été rédigés afin de :

- définir la structure de la base de données
- assurer l'intégrité des données via des clés primaires, clés étrangères et contraintes NOT NULL
- faciliter l'installation du projet sur un nouvel environnement

Annexe III.2 : script de création des tables

Les contraintes appliquées incluent :

- la clé primaire sur l'identifiant utilisateur
- l'unicité de l'email
- des champs obligatoires (NOT NULL)
- des valeurs par défaut pour certains champs
- relations entre tables via les clés étrangères pour garantir l'intégrité référentielle

Outre la table utilisateurs, la base comprend également des tables *trajets*, *véhicules*, *réservations*, *passagers*, etc., interconnectées, pour assurer la cohérence et la relation entre les données.

2. Jeu de données

Annexe III.3 : script de données seed

Un script seed a été créé afin de faciliter :

- les tests fonctionnels
- les démonstrations devant le jury
- la validation du bon fonctionnement des fonctionnalités

Ce jeu de données comprend :

- plusieurs utilisateurs avec des rôles différents (passager, conducteur, passager-conducteur)
- des mots de passe hachés avec Bcrypt pour garantir la sécurité même pour les données de démonstration
- des données réalistes pour chaque utilisateur (emails, dates, biographie, etc.)

L'utilisation de ce script seed permet de présenter des cas concrets et réalistes au jury tout en évitant l'utilisation de données personnelles.

3. NoSQL

Annexe III.4 : capture d'écran de logs JSON

MongoDB a été la solution envisagée en premier lieu ; un cluster a été créé, la base a été configurée avec ses collections, la librairie a été installée via Composer. Cependant, l'extension PHP de mongoDB n'était pas compatible avec le PHP embarqué dans XAMPP sur macOS.

Pour remédier à cela, un système de fichier JSON a été mis en place. Cette approche respecte la logique du NoSQL et est architecturalement équivalente à MongoDB :

documents autonomes, schémas flexibles, données non normalisées. Chaque entrée est un objet JSON indépendant pouvant contenir des informations variables selon le contexte.

Chaque log est enregistré avec la structure :

```
{ "id": "log_69cc11434653b9.53369485",  
  "date": "2026-03-31 20:24:03",  
  "niveau": "INFO",  
  "action": "recherche_trajet",  
  "message": "Recherche : Bordeaux → Mussidan le 2026-09-22 (0 résultat(s))",  
  "utilisateur": 19,  
  "ip": "::1",  
  "page": "\eco_ride\PROJET\UTILISATEUR\USR-recherche_trajet.php?departure=B  
ordeaux&destination=Mussidan&date=2026-09-22&passenger=1" }
```

Fonctions PHP (logs.php)

Quatre fonctions constituent la bibliothèque des logs :

- logAction(\$action, \$message, \$niveau, \$userId) : écrit un log avec verrou exclusif (flock) pour éviter toute corruption en cas d'accès simultané
- getLogs(\$niveau, \$action, \$userId, \$limite, \$offset) : lecture avec filtres optionnels et pagination
- countLogs() : comptage pour la pagination
- clearLogs() : vidage complet, réservé à l'administrateur

Intégration

Annexe III.5 : page admin logs

La fonction logAction() est appelée à chaque action significative : recherche, modification, suppression d'un trajet, connexion, échecs, etc.

La page ADM-logs.php offre un tableau de bord avec compteurs par niveau (INFO / WARNING), des filtres par type d'action, et bouton de suppression protégé par confirmation.

Sécurité :

- Accès au fichier JSON bloqué via .htaccess (HTTP 403 Forbidden)
- Verrou exclusif (flock) à chaque écriture pour prévenir la corruption
- Toutes les données affichées sont échappées via htmlspecialchars()
- Permissions système : fichier accessible en écriture uniquement par le processus Apache (utilisateur daemon), via chown et chmod 664

Exemple concret - connexion d'un utilisateur

Pour illustrer la cohérence des données entre MySQL et JSON, voici le résultat d'une connexion :

L'utilisateur se connecte → la vérification des identifiants est effectuée dans la table utilisateurs, et la session est initialisée côté serveur.

Annexe III.6 : table utilisateurs

La connexion est simultanément enregistrée dans logs.json

Annexe III.7 : logs de connexion dans le fichier JSON

Cette cohérence entre les deux systèmes (MySQL pour l'authentification, JSON pour l'historique des actions) garantit la traçabilité complète des connexions.

IV. Déploiement

1. Préparation de l'environnement

1.1 *Configuration du serveur*

Utilisation de Docker

Docker permet de simuler un environnement serveur en local, afin de préparer le déploiement et anticiper les problèmes potentiels. Il permet également de reproduire exactement l'environnement de développement (versions PHP, MySQL, etc.) sur n'importe quelle machine, ce qui facilitera la démonstration du projet à distance.

Installation

Docker Desktop a été téléchargé depuis le site officiel. Pour vérifier que l'installation s'est correctement effectuée, la commande suivante a été effectuée dans le terminal :

```
docker --version
```

```
docker run hello-world
```

La réponse obtenue confirme le bon fonctionnement :

```
Hello from Docker!
```

```
This message shows that your installation appears to be working correctly.
```

Configuration et test sur environnement Windows

La configuration complète de Docker a été réalisée et testée sur un environnement Windows.

Le dépôt GitHub a d'abord été cloné.

Annexe IV.1 : extrait docker-compose.yml

Le projet inclut un docker-compose.yml définissant trois services :

- php : serveur Apache + PHP 8, exposé sur port 8080
- mysql : base de données MySQL 8, exposée sur le port 3307
- phpmyadmin : interface d'administration sur le port 8081

Un Dockerfile configure également les extensions PHP nécessaires, notamment PDO MySQL.

Le fichier connexion.php a été adapté pour fonctionner sur les deux environnements :

- Sur Mac avec XAMPP → utilise 127.0.0.1
- Dans Docker → utilise la variable d'environnement DB_HOST=mysql

Lancement

Annexe IV.2 : screenshot du site accessible sur le localhost 8080 et 8081

docker-compose up -d

Après import de la base de données via phpMyAdmin et création des fichiers JSON avec les bonnes permissions, le site était accessible sur <http://localhost:8080>

2. Procédure de déploiement

2.1 *Transfert des fichiers*

Annexe IV.3 : screenshot du projet déployé sur Railway

Le déploiement a été effectué sur Railway. Le dépôt GitHub est connecté directement à Railway, qui récupère automatiquement le code source. Les variables d'environnement (DB_HOST, DB_NAME, DB_USER, DB_PASSWORD) sont configurés dans le tableau de bord Railway.

Annexe IV.4 : extrait du [README.md](#)

Le [README.md](#) est disponible à la racine du dépôt GitHub décrivant les étapes nécessaires pour déployer le projet en local (XAMPP ou Docker) ou sur Railway.

2.2 *Importation de la base de données*

Annexe IV.5 : screenshot de la base de données sur Railway

La base de données MySQL a été créée via le service MySQL de Railway. Le fichier eco_ride.sql a été importé via l'interface Railway. Le fichier connexion.php a été adapté pour utiliser les variables d'environnement Railway.

3. Tests post-déploiement

Une fois le déploiement effectué, plusieurs tests ont été réalisés afin de vérifier le bon fonctionnement de l'application en conditions réelles :

- accès au site via l'URL
- connexion avec les différents rôles (admin, employé, utilisateur)
- recherche et réservation d'un trajet

- vérification de l'accès à la base de données
- vérification du HTTPS activé automatiquement

V. Conclusion : évolutions et perspectives du projet

EcoRide est mon premier projet fullstack, et de cette ampleur. Il m'a appris qu'il faut être organisé et anticiper, tout en sachant s'adapter quand les contraintes arrivent en cours de route.

Parmi les principales contraintes que j'ai rencontrées durant le projet : la compatibilité de MongoDB avec XAMPP sur Mac, qui m'a amenée à revoir la manière dont je devais intégrer du NoSQL. La configuration Docker a également posé problème sur Mac, résolue en basculant sur Windows, et enfin, la mise en place de l'upload de photo de profil qui a nécessité des modifications dans le fichier php.ini afin d'autoriser les bons types de fichiers et les tailles suffisantes.

Je suis particulièrement fière d'avoir surmonté ma peur du back-end et du NoSQL, et d'avoir trouvé des solutions adaptées à mon environnement.

Rétrospective

Keep : je vais continuer la séparation HTML/CSS/JS/PHP, et faire des branches Git par fonctionnalités

Improve : je vais améliorer la propreté du code, la gestion du temps, le nommage des branches

Start : je vais commencer à me faire confiance, à penser global dès le début (maquettes, BDD, architecture), à aller à l'essentiel

Stop : je vais arrêter d'éviter les parties qui font peur, de développer des fonctionnalités non prioritaires avant les essentielles

Évolutions futures

- Géolocalisation via Google Maps API pour proposer des trajets proches géographiquement
- Partie NoSQL sur MongoDB une fois l'environnement compatible
- Protection CSRF en production

ANNEXES

I. Analyse

Annexe I.1 : Wireframe web de la page d'accueil
Annexe I.2 : Wireframe mobile de la page d'accueil
Annexe I.3 : Mockup web de la page d'accueil
Annexe I.4 : Mockup mobile de la page d'accueil
Annexe I.5 : Wireframe web de la page proposer un trajet
Annexe I.6 : Wireframe mobile de la page proposer un trajet
Annexe I.7 : Mockup web de la page proposer un trajet
Annexe I.8 : Mockup mobile de la page proposer un trajet
Annexe I.9 : Wireframe web de la page de résultat de recherche
Annexe I.10 : Wireframe mobile de la page résultat de recherche
Annexe I.11 : Mockup web de la page résultat de recherche
Annexe I.12 : Mockup mobile de la page résultat de recherche
Annexe I.13 : Diagramme de cas d'utilisation
Annexe I.14 : Diagramme de séquence - Connexion utilisateur
Annexe I.15 : Diagramme de classe UML
Annexe I.16 : Screenshot des tâches Trello
Annexe I.17 : Screenshot des commentaires dans le code

II. Développement

Annexe II.1 : Screenshot de la page Proposer un trajet
Annexe II.2 : Screenshot de la page Recherche de trajet
Annexe II.3 : Exemples d'éléments responsives
Annexe II.4 : Screenshot de l'animation JavaScript
Annexe II.5 : Screenshots des pages de la partie Admin
Annexe II.6 : Screenshots des pages de la partie Employé
Annexe II.7 : Screenshot de la page Proposer un trajet remplie
Annexe II.8 : Screenshot de la page Proposer un trajet 2
Annexe II.9 : Screenshot de la page de confirmation

Annexe II.10 : Extrait de code du formulaire de la page Proposer un trajet

Annexe II.11 : Extrait du code PHP de récupération des données

Annexe II.12 : Extrait du code JavaScript de la récupération des arrêts

Annexe II.13 : Extrait du code PHP de l'insertion SQL

Annexe II.14 : Extrait du code JavaScript de l'ajout dynamique des arrêts

Annexe II.15 : Screenshot de la page Mes trajets

Annexe II.16 : Screenshot de la page Détails trajet

Annexe II.17 : Extrait du code PHP de la récupération des trajets

Annexe II.18 : Extrait du code PHP du tri des trajets

Annexe II.19 : Extrait du code PHP de l'affichage des trajets

Annexe II.20 : Extrait du code PHP de la vérification du propriétaire

Annexe II.21 : Extrait du code PHP du modal de modification

III. Base de données

Annexe III.1 : Diagramme de la base de données eco-ride

Annexe III.2 : Extrait du script SQL de la création des tables

Annexe III.3 : Extrait du script SQL de l'insertion des données seed

Annexe III.4 : Extrait des logs dans le fichier JSON

Annexe III.5 : Screenshot des logs affichés sur la page admin

Annexe III.6 : Screenshot des tables trajets_passagers et credits

Annexe III.7 : Extrait des transactions dans le fichier JSON

IV. Déploiement

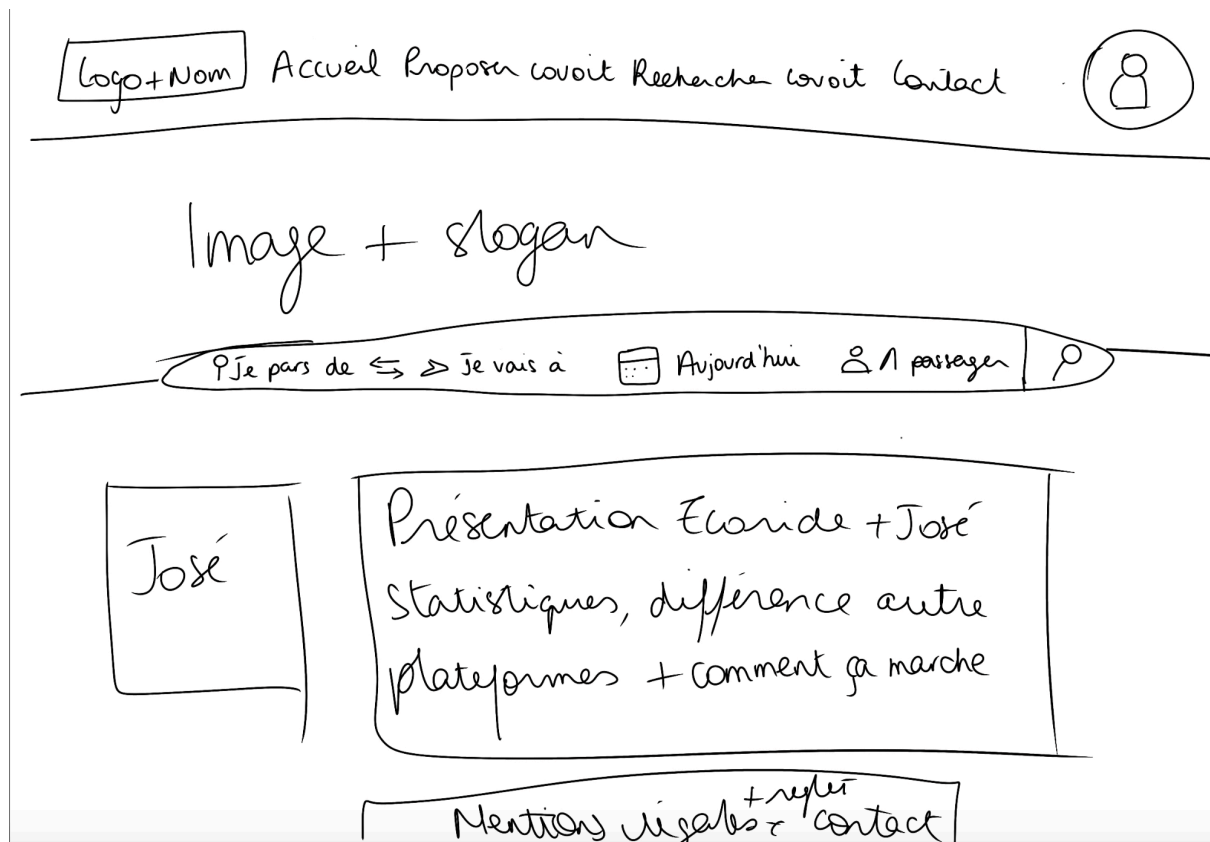
Annexe IV.1 : Extrait du docker-compose.yml

Annexe IV.2 : Screenshot du site accessible avec Docker

Annexe IV.3 : Screenshot du projet déployé sur Railway

Annexe IV.4 : Screenshot du README.md

Annexe I.1 : Wireframe web de la page d'accueil



Annexe I.2 : Wireframe mobile de la page d'accueil

logo

≡

8

Image+slogan

Rechercher un trajet

je pars de ...

je vais à ...

23/03/25 81 Q

José

Présentation
Ecoride + José
Fonctionnement

Avis

Superbe plateforme

- ANTOINE

Très pratique

- NINA

Mentions légales GGU Contact

Annexe I.3 : Mockup web de la page d'accueil

 **Eco Ride**

[Accueil](#) [Covoiturage](#) [Aide](#) 



Roulons vers un futur plus vert

Départ

Arrivée

Date

Passagers

Ville de départ

Ville d'arrivée

21/05/2025

1

Comment ça marche ?

1. Recherchez votre trajet
2. Choisissez votre conducteur
3. Réservez, vous pouvez covoiturer

Prêt.e à covoiturer ?

Créez un compte

[Mentions légales](#) [Cookies](#) [Règlement](#) [Newsletter](#) [Réseaux sociaux](#)

Annexe I.4 : Mockup mobile de la page d'accueil

Eco Ride



Roulons vers un futur plus vert

Départ

Ville de départ

Arrivée

Ville d'arrivée

Date

21/05/2025

Passagers

1

Rechercher

Comment ça marche ?

1. Recherchez votre trajet
2. Choisissez votre conducteur
3. Réservez, vous pouvez covoiturer

Prêt.e à covoiturer ?

Créez un compte

Mentions légales

Cookies

Règlement

Newletter

Réseaux sociaux

Annexe I.5 : Wireframe web de la page proposer un trajet

Logo+Nom

Accueil Proposer un trajet Recherche un trajet Contact


No C

Votre trajet

Etape 1/2 *obligatoire

D'où partent-nous? *
Adresse départ

Où allons nous? *

Arrêt n°1
l'arrêt +/- sur la route
⊕ rajouter un arrêt

Nb passagers possibles *

1

Le trajet vous fera gagner 10€ par passagers une fois le trajet terminé en bonnes conditions.

Véhicule utilisé *
choisir ✓

Date départ *
JJ/MM/AAAA
Heure *
HH:MM

Ajouter des précisions
Cp libre

Valider

Annexe I.6 : Wireframe mobile de la page proposer un trajet

logo

≡

8

Votre trajet

D'où partons nous? *

+

Ajouter un arrêt

Véhicule utilisé *

v choisir

Date départ *

JJ / MM / AAAA

Heure *

HH : MM

Où allons-nous? *

Nb passagers *

1

▲▼

Le trajet fera
gagner 10 crédits
une fois terminé.

Précisions (optionnel)

Valider

Mentions légales GGU Contact

Annexe I.7 : Mockup web de la page proposer un trajet

 **Eco Ride**

[Accueil](#) [Covoiturage](#) [Aide](#) 

Proposer un trajet

Départ

Adresse de départ

Ajouter un arrêt

Véhicule

Date du départ

Heure de départ

Arrivée

Adresse d'arrivée

Nombre de places disponibles

Autres précisions (optionnel)

Ex : passage par autoroute

Valider

[Mentions légales](#) [Cookies](#) [Règlement](#) [Newsletter](#) [Réseaux sociaux](#)

Annexe I.8 : Mockup mobile de la page proposer un trajet

Eco Ride

Proposer un trajet

Départ

Adresse de départ

Ajouter un arrêt

Véhicule

Date du départ

Heure de départ

Arrivée

Adresse d'arrivée

Nombre de places disponibles

Autres précisions (optionnel)

Valider

[Mentions légales](#)[Cookies](#)[Règlement](#)[Newsletter](#)[Réseaux sociaux](#)

Annexe I.9 : Wireframe web de la page de résultat de recherche

Logo+Nom

Accueil Proposer un voitur Recherche Voiture Contact

9 Bordeaux

↔

➤ Périgueux

Sam. 12 avr.

2 1 passager

Filterer

Tout effacer

€

Prix max.

Trajet écologique ☒

Note chauffeur min

~~Durée max voyage~~
↓
~~demander au chauffeur~~

12h00

11h00

13h00

Bordeaux

Périgueux

5€

Thierry ★ 4,7 | 1 place restante

Voyage écologique | + détails

Idem

Idem

Jour précédent

Jour suivant

Annexe I.10 : Wireframe mobile de la page résultat de recherche

Logo

≡

8

Rechercher un trajet

Bordeaux

Paris

26/05/25 8 1 Q

Filter

Bordeaux → Périgueux

12h00 → 13h00

8 Antoine
4,7 ☆

Details

Idem

Idem

Mentions légales GGU Contact

Annexe I.11 : Mockup web de la page résultat de recherche

 **Eco Ride**

[Accueil](#) [Covoiturage](#) [Aide](#) 

Départ

Bordeaux

Arrivée

Bayonne

Date

21/05/2025

Passagers

1

Rechercher

Filtres

Prix maximum

Ex : 20

Note chauffeur minimale

Ex : 4.5

Trajet écologique

☒ Oui ☐ Non

Bordeaux → Bayonne

Départ : 8h30

Place.s : 2

Durée de trajet estimée : 2h

Chauffeur



Nino C.

5 ☆

[Voir les détails](#)

[Mentions légales](#) [Cookies](#) [Règlement](#) [Newsletter](#) [Réseaux sociaux](#)

Annexe I.12 : Mockup mobile de la page résultat de recherche

Eco Ride

Départ

Bordeaux

Arrivée

Bayonne

Date

21/05/2025

Passagers

1

Rechercher

Filtres

Bordeaux → Bayonne

Départ : 8h30

Durée de trajet estimée : 2h

Place.s : 2

Chauffeur



Nino C.

5 ☆

[Voir les détails](#)

Mentions légales

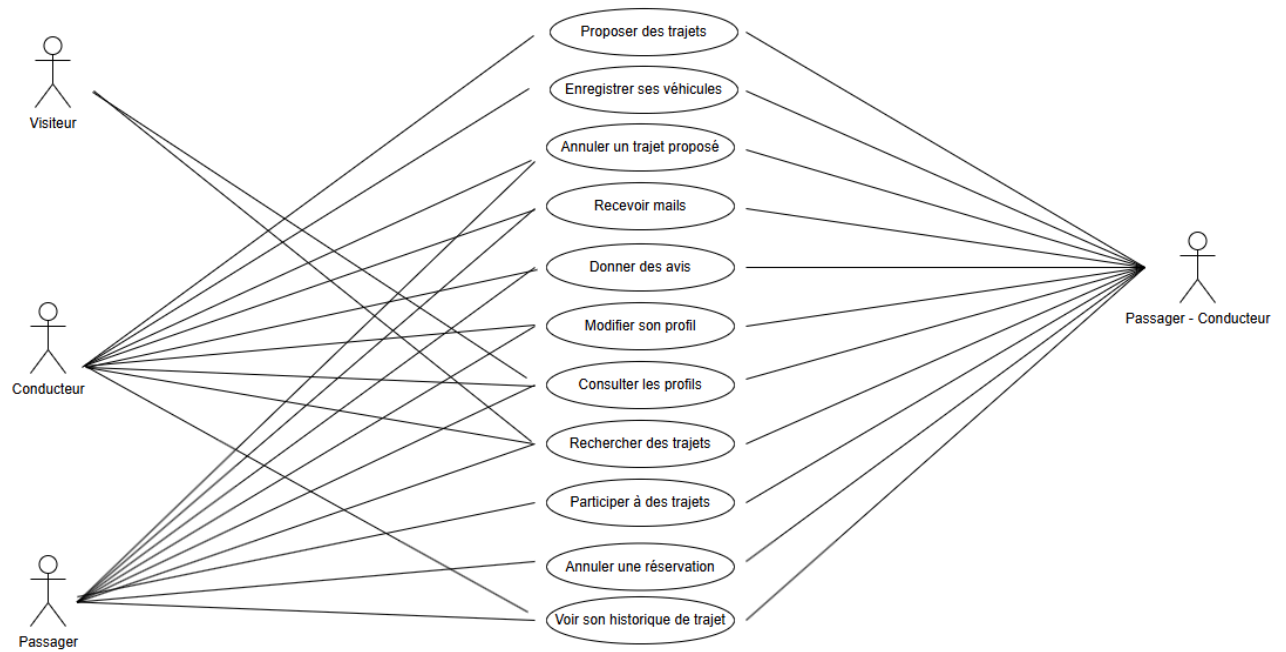
Cookies

Règlement

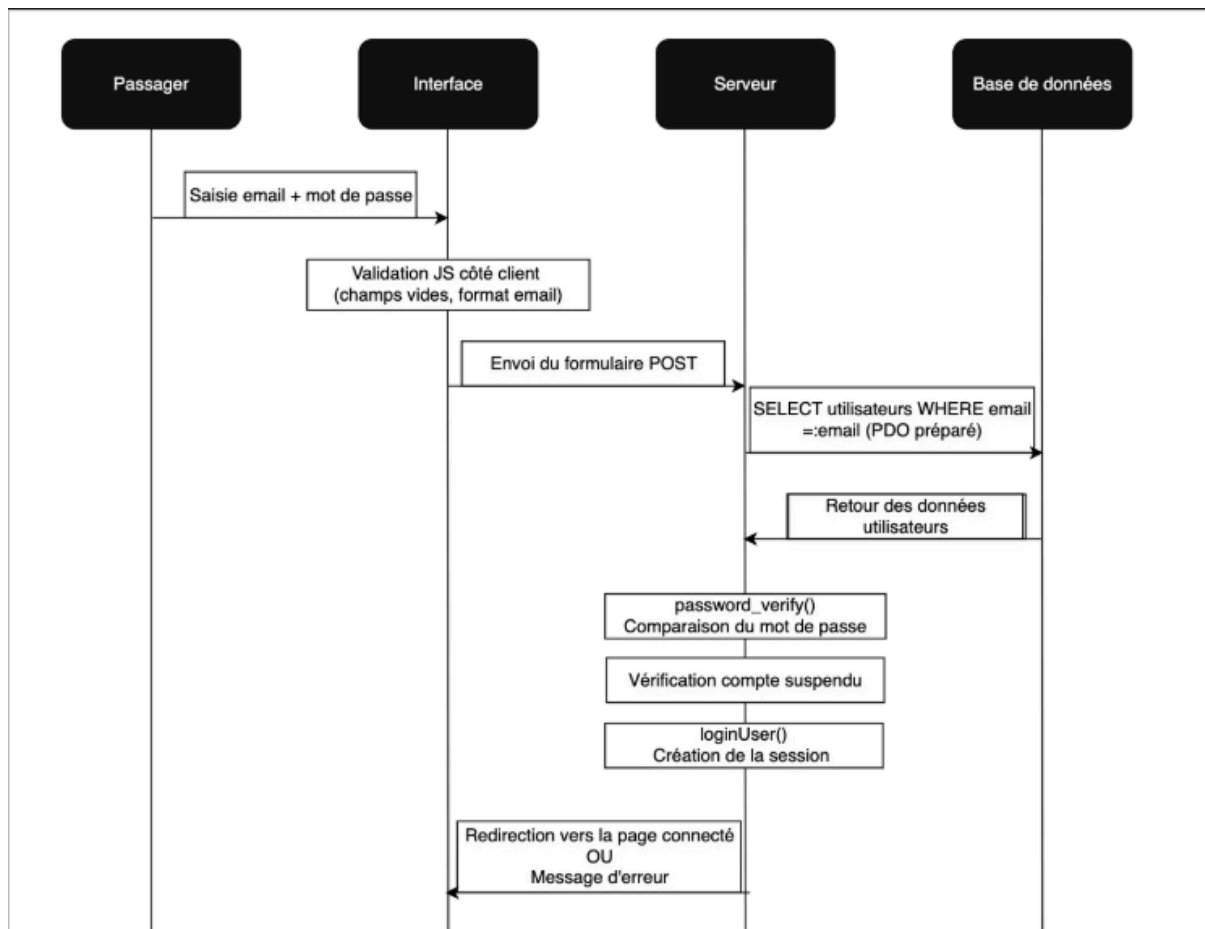
Newletter

Réseaux sociaux

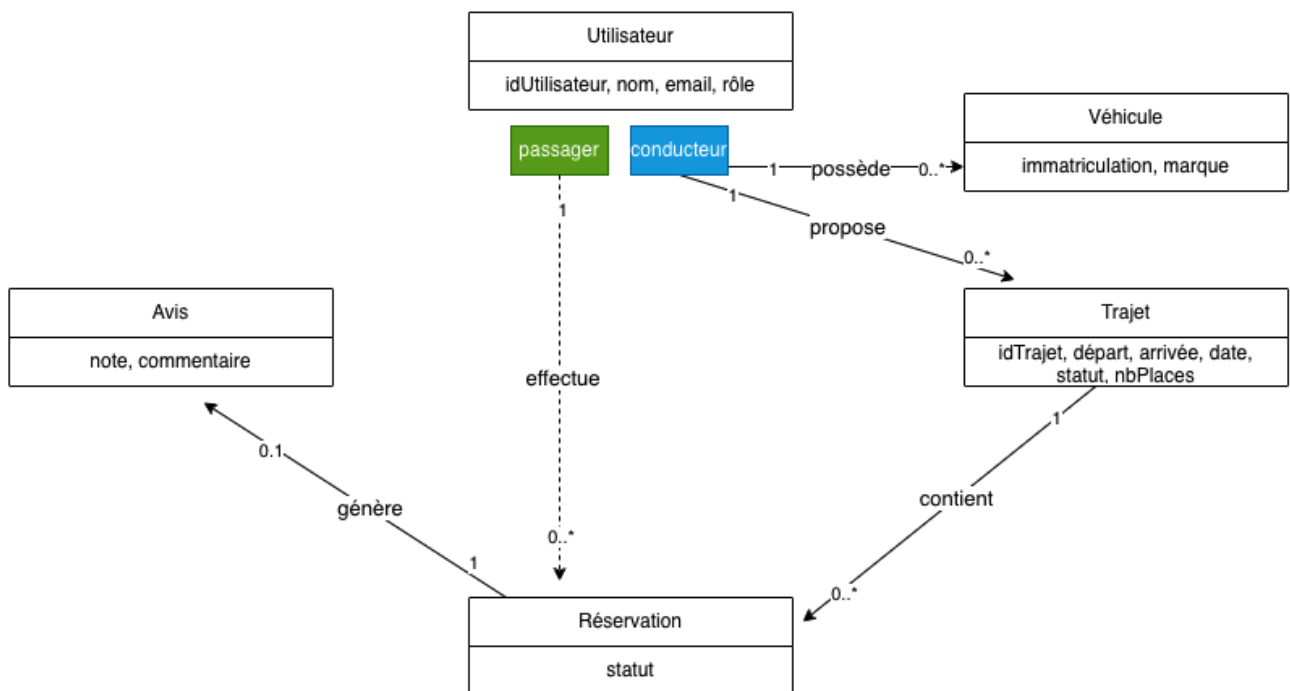
Annexe I.13 : Diagramme de cas d'utilisation



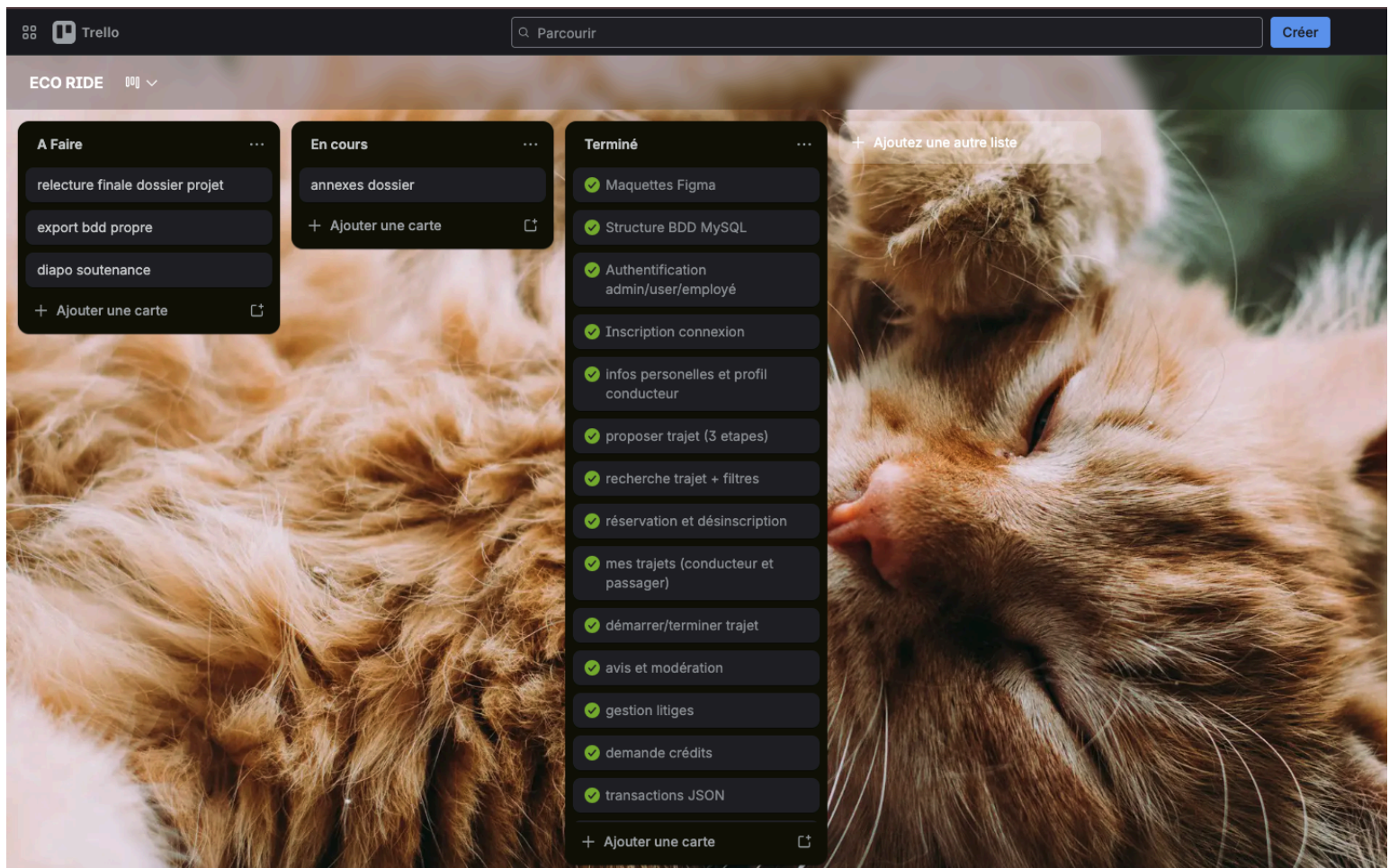
Annexe I.14 : Diagramme de séquence - Connexion utilisateur



Annexe I.15 : Diagramme de classe UML



Annexe I.16 : Screenshot des tâches Trello



Annexe I.17 : Screenshot des commentaires dans le code

```
<?php
include('../PHP/auth.php'); // Démarre la session et charge les fonctions
requireLogin(); // Redirige si l'utilisateur n'est pas connecté
?>
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title> Mon compte - Aide </title>
    <link rel="stylesheet" href="../CSS/style_global.css">
    <link rel="stylesheet" href="../CSS/CSS UTILISATEUR/USR-aide.css">

    <link href="https://fonts.googleapis.com/css2?family=Montserrat:wght@400;700&family=Quicksand:wght@400;600&display=swap" rel="stylesheet">
</head>
<body>
<!-- Header commun -->
<?php include('../COMPONENTS/COMP-header.php') ; ?>
<main>

<section>
    <h2>Aide & Support</h2>

    <!-- Introduction de la section d'aide -->
    <p>Bienvenue dans la section d'aide. Voici les réponses aux questions les plus fréquentes. Si vous ne trouvez pas votre réponse, vous pouvez nous contacter.</p>

<!-- Bloc FAQ -->
<div class="faq-section">
    <h3>F.A.Q.</h3>
    <h3>Questions fréquentes</h3>

    <div class="faq-question">
        <p><strong>Comment modifier mon mot de passe ?</strong></p>
        <p>Allez dans "Mon profil" puis cliquez sur "Modifier le mot de passe".</p>
    </div>

    <div class="faq-question">
        <p><strong>Comment supprimer le trajet ?</strong></p>
        <p>Rendez-vous dans la section "Mes trajets", cliquez sur le trajet concerné puis sélectionnez "Supprimer".</p>
    </div>

    <div class="faq-question">
        <p><strong>Comment contacter un autre utilisateur ?</strong></p>
        <p>Utilisez la messagerie intégrée dans votre espace personnel.</p>
    </div>
</div>

</main>

<!-- Footer commun -->
<?php include('../COMPONENTS/COMP-footer.html'); ?>
<script src="../JS/USR-aide.js"></script>
```

Annexe II.1 : Screenshot de la page Proposer un trajet

 Eco Ride

[Accueil](#) [Proposer un trajet](#) [Rechercher un covoiturage](#) 

Votre trajet - Étape 1 sur 2

* Champs obligatoires

Départ

Adresse de départ *

Arrêts (optionnel)

+ Ajouter un arrêt

Date de départ *

Heure de départ *

Arrivée

Adresse d'arrivée *

Heure d'arrivée *

Nombre de places disponibles *

Prix par passager (en crédits) *

 2 crédits sont retenus par la plateforme.
Minimum : 2 crédits.

Vous gagnerez 0 crédits par passager (après retenue plateforme).

Informations pratiques

Véhicule utilisé *

-- Sélectionnez un véhicule --

Point de rendez-vous, lieu de dépose et autres précisions *

Ex : départ devant la gare Saint-Jean côté taxis, dépose Place Gambetta devant le café, coffre petit...

 Ces informations sont indispensables pour que vos passagers puissent vous retrouver.

Étape suivante →

Annexe II.2 : Screenshot de la page Recherche de trajet

 **Eco Ride**

[Accueil](#) [Proposer un trajet](#) [Rechercher un covoiturage](#) 

Je pars de

Agen

Je vais à

Niort

Date

22/06/2026



Passagers

1



Filtrer

Tout effacer

☒ Trajet écologique 

Prix maximum (crédits)

Durée maximum (heures)

ex: 2.5

Note chauffeur minimale



Agen → Niort

Passant par : Bergerac

22/06/2026

 Départ : 11:00

 Arrivée : 15:00

Clara Soual

Aucun avis

 Durée : 4h00

 3 place(s)

 2 crédit(s)

 Trajet écologique

Voir détails

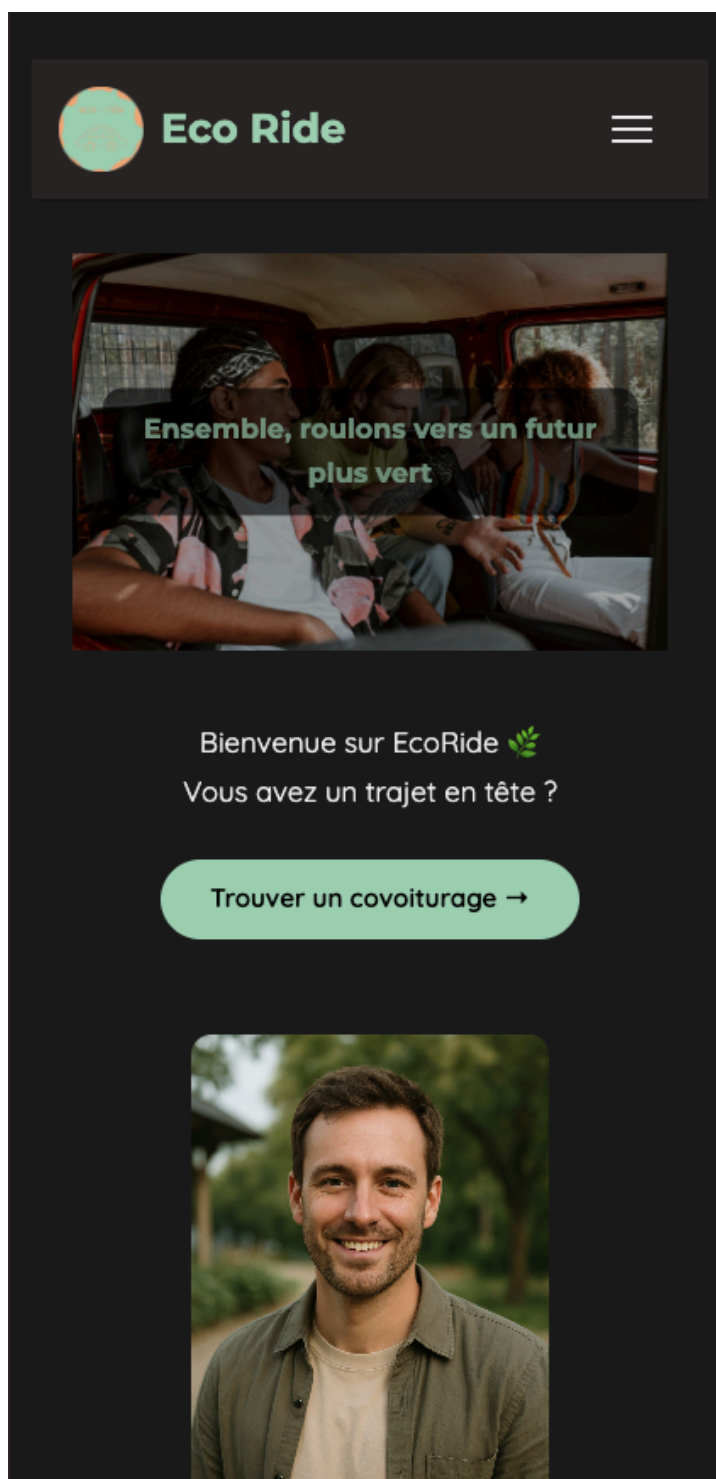
[Mentions légales](#) [Règlement de la plateforme](#) [Déconnexion](#)

ecoride@contact.com

© 2025 ECO RIDE. Tous droits réservés

Annexe II.3 : Exemples d'éléments responsives

Index :



Recherche trajet :

 **Eco Ride** 

Je pars de

Je vais à

Date

Passagers



 **Filtrer les résultats**

 **Antoine B.** ★ 5.0/5

Proposer trajet :

 **Eco Ride** 

Votre trajet - Étape 1 sur 2

* Champs obligatoires

 **Départ**

Adresse de départ *

Ville ou adresse de départ

Arrêts (optionnel)

Arrêt n°1

+ Ajouter un arrêt

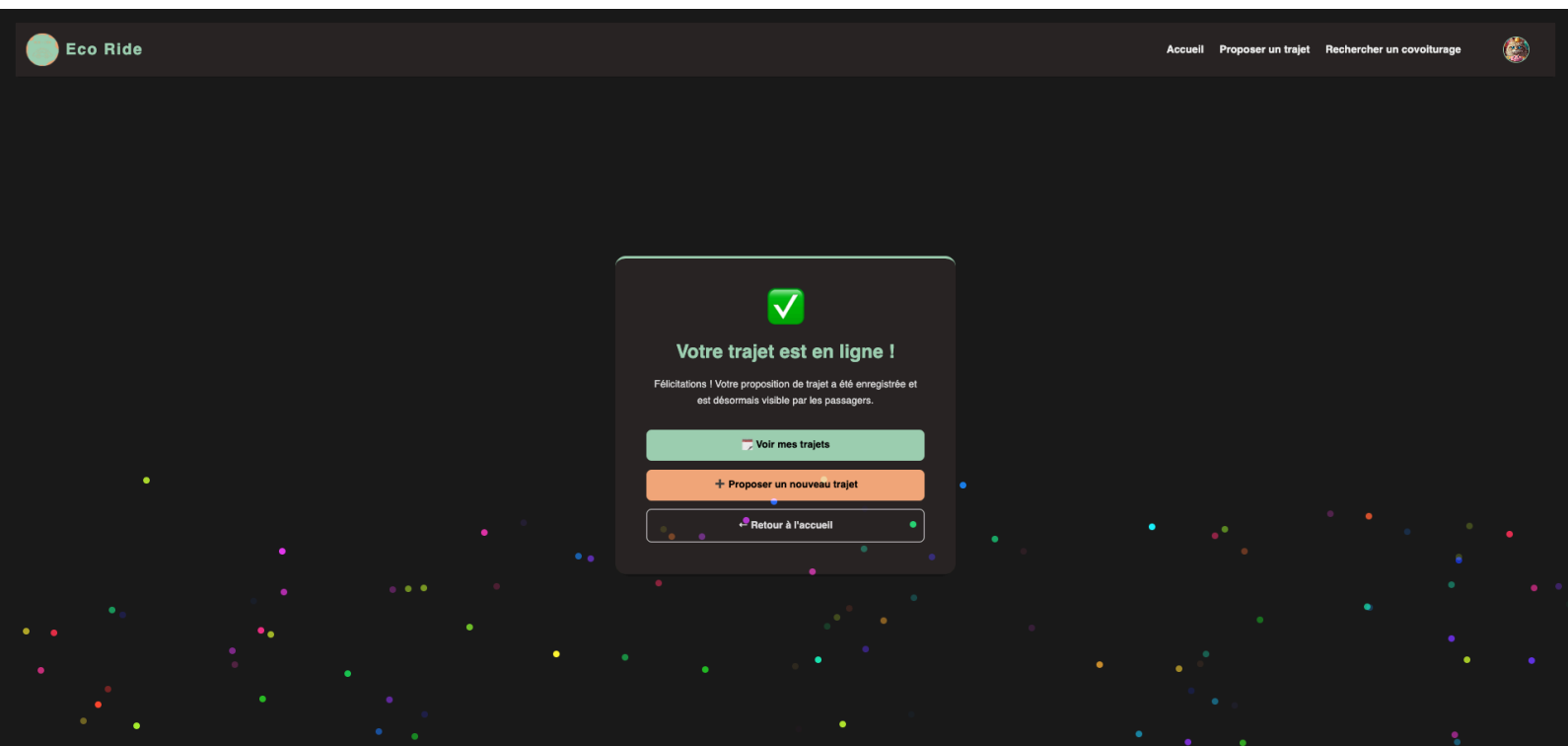
Date de départ *

jj/mm/aaaa 

Heure de départ *

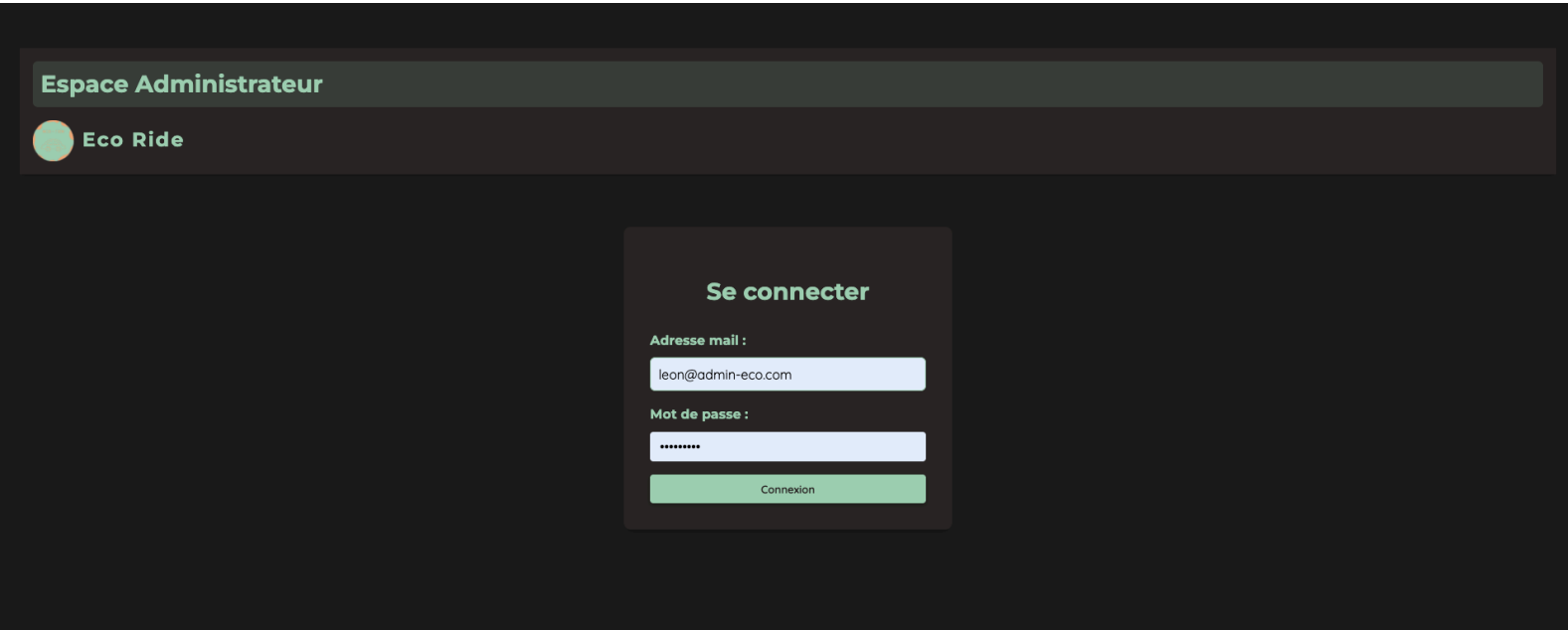
--:-- 

Annexe II.4 : Screenshot de l'animation JavaScript

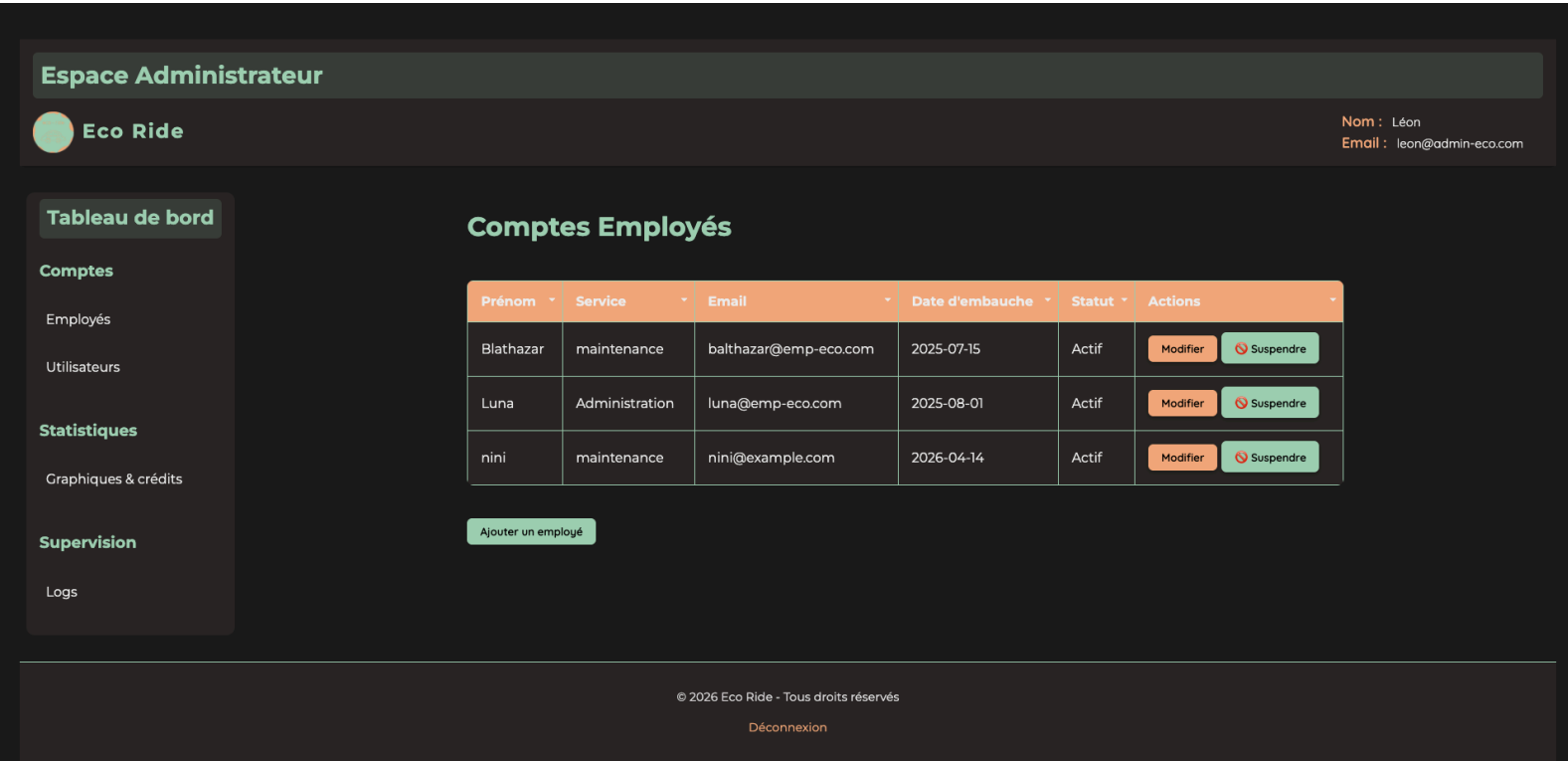


Annexe II.5 : Screenshots des pages de la partie Admin

Page de connexion :



Page gestion des employés :



Page gestion des utilisateurs :

Espace Administrateur

Eco Ride

Nom : Léon
Email : leon@admin-eco.com

Tableau de bord

Comptes

Employés

Utilisateurs

Statistiques

Graphiques & crédits

Supervision

Logs

Compte Utilisateurs

Nom, mail...

Rechercher

Prénom	Nom	Email	Date d'inscription	Statut	Actions
Nino	C.	nino@example.com	2025-11-06 16:57:52	Actif	Voir Suspendre
Finn	M.	finn@example.com	2025-11-06 16:57:52	Actif	Voir Suspendre
Antoine	B.	antoine@example.com	2025-11-06 16:57:52	Actif	Voir Suspendre
Nina	R.	nina@example.com	2025-11-06 16:57:52	Actif	Voir Suspendre
Théo	K.	theo@example.com	2025-11-06 16:57:52	Actif	Voir Suspendre
Jeanne	S.	jeanne@example.com	2025-11-06 16:57:52	Actif	Voir Suspendre
Clara	Soual	clara.soual@example.com	2026-01-22 15:23:09	Actif	Voir Suspendre
testcredits	testcredits	test.credits@example.com	2026-04-06 13:42:16	Actif	Voir Suspendre
testprofil	profiltest	testprofil@example.com	2026-04-15 17:01:23	Actif	Voir Suspendre

© 2026 Eco Ride - Tous droits réservés

Déconnexion

Page graphiques et crédits :

Espace Administrateur

Eco Ride

Nom : Léon
Email : leon@admin-eco.com

Tableau de bord

Comptes

Employés

Utilisateurs

Statistiques

Graphiques & crédits

Supervision

Logs

Statistiques

Vue d'ensemble de l'activité de la plateforme

De

24/03/2026

au

23/04/2026

Appliquer

COVOITURAGES TERMINÉS

0

sur la période sélectionnée

Covoiturages par jour

1

0

24/0327/0330/0302/0405/0408/0411/0414/0417/0420/0423/04

CRÉDITS GAGNÉS PAR LA PLATEFORME

8

sur la période sélectionnée

Crédits gagnés par la plateforme

4

3

2

1

0

24/0327/0330/0302/0405/0408/0411/0414/0417/0420/0423/04

© 2026 Eco Ride - Tous droits réservés

Déconnexion

Annexe II.6 : pages de la partie employé

Page de gestion des avis :

Espace Employé

Eco Ride

Nom : Blathazar
Email : balthazar@emp-eco.com

Tableau de bord

Modération

Avis à valider

Litiges

Covoiturages signalés

Crédits

Demandes de crédits

Avis utilisateurs - Modération

Afficher :

En attente

ID	Date	Auteur	Conducteur	Trajet	Note	Commentaire	Actions
12	16/03/2026	Antoine B.	Nina R.	Béziers → Bordeaux	5.0/5	Nina a été une passagère agréable.	Valider Refuser
11	16/03/2026	Nina R.	Antoine B.	Béziers → Bordeaux	3.5/5	Je ne me suis pas sentie à l'aise.	Valider Refuser

© 2026 Eco Ride - Tous droits réservés

Déconnexion

Page de signalement des covoiturages :

Espace Employé

Eco Ride

Nom : Blathazar
Email : balthazar@emp-eco.com

Tableau de bord

Modération

Avis à valider

Litiges

Covoiturages signalés

Crédits

Demandes de crédits

Covoiturages signalés — Passager signale un conducteur

N° trajet	Trajet	Date	Passager	Conducteur	Statut	Détails	Actions
4	B77ziers → Bordeaux	15/03/2026 à 14:00:00	Nina R.	Antoine B.	Traité	Voir détails	Crédits bloqués

Signalements passagers — Conducteur signale un passager

Aucun signalement conducteur → passager.

© 2026 Eco Ride - Tous droits réservés

Déconnexion

Page de gestion des demandes de crédits :

Espace Employé



Nom : Blathazar
Email : balthazar@emp-eco.com

Tableau de bord

Modération

Avis à valider

Litiges

Covoiturages signalés

Crédits

Demandes de crédits

Demandes de crédits

Afficher : En attente

Date	Utilisateur	Email	Crédits accordés	Statut	Actions
06/04/2026 14:02	testcredits testcredits	test.credits@example.com	20 crédits	En attente	Accepter Refuser
06/04/2026 14:04	testcredits testcredits	test.credits@example.com	20 crédits	En attente	Accepter Refuser

Cf. Annexe II.7 : page proposer un trajet

 **Eco Ride**

[Accueil](#) [Proposer un trajet](#) [Rechercher un covoiturage](#) 

Votre trajet - Étape 1 sur 2

* Champs obligatoires

 **Départ**

Adresse de départ *

Mérignac

Arrêts (optionnel)

Arrêt n°1

+ Ajouter un arrêt

Date de départ *

12/07/2026

Heure de départ *

11:00

 **Arrivée**

Adresse d'arrivée *

Montpon

Heure d'arrivée *

12:00

Nombre de places disponibles *

4

Prix par passager (en crédits) *

 2 crédits sont retenus par la plateforme.
Minimum : 2 crédits.

5

Vous gagnerez 3 crédits par passager
(après retenue plateforme).

 **Informations pratiques**

Véhicule utilisé *

Toyota Corolla Bleu

Point de rendez-vous, lieu de dépose
et autres précisions *

Départ Carrefour Mérignac soleil,
arrivée gare Montpon

 Ces informations sont indispensables pour
que vos passagers puissent vous retrouver.

Étape suivante →

[Mentions légales](#) [Règlement de la plateforme](#) [Déconnexion](#)
ecoride@contact.com

© 2025 ECO RIDE. Tous droits réservés

Annexe II.8 : page proposer trajet 2

 Eco Ride

[Accueil](#) [Proposer un trajet](#) [Rechercher un covoiturage](#) 

Résumé de votre trajet - Étape 2 sur 2

Vérifiez les informations de votre trajet avant de le confirmer.

Départ

ADRESSE
Mérignac

DATE
12/08/2026

HEURE DE DÉPART
10:00

VÉHICULE
Toyota Corolla Bleu

Arrivée

ADRESSE
Montpon

HEURE D'ARRIVÉE
11:00

PLACES DISPONIBLES
3

PRIX PAR PASSAGER
5 crédits

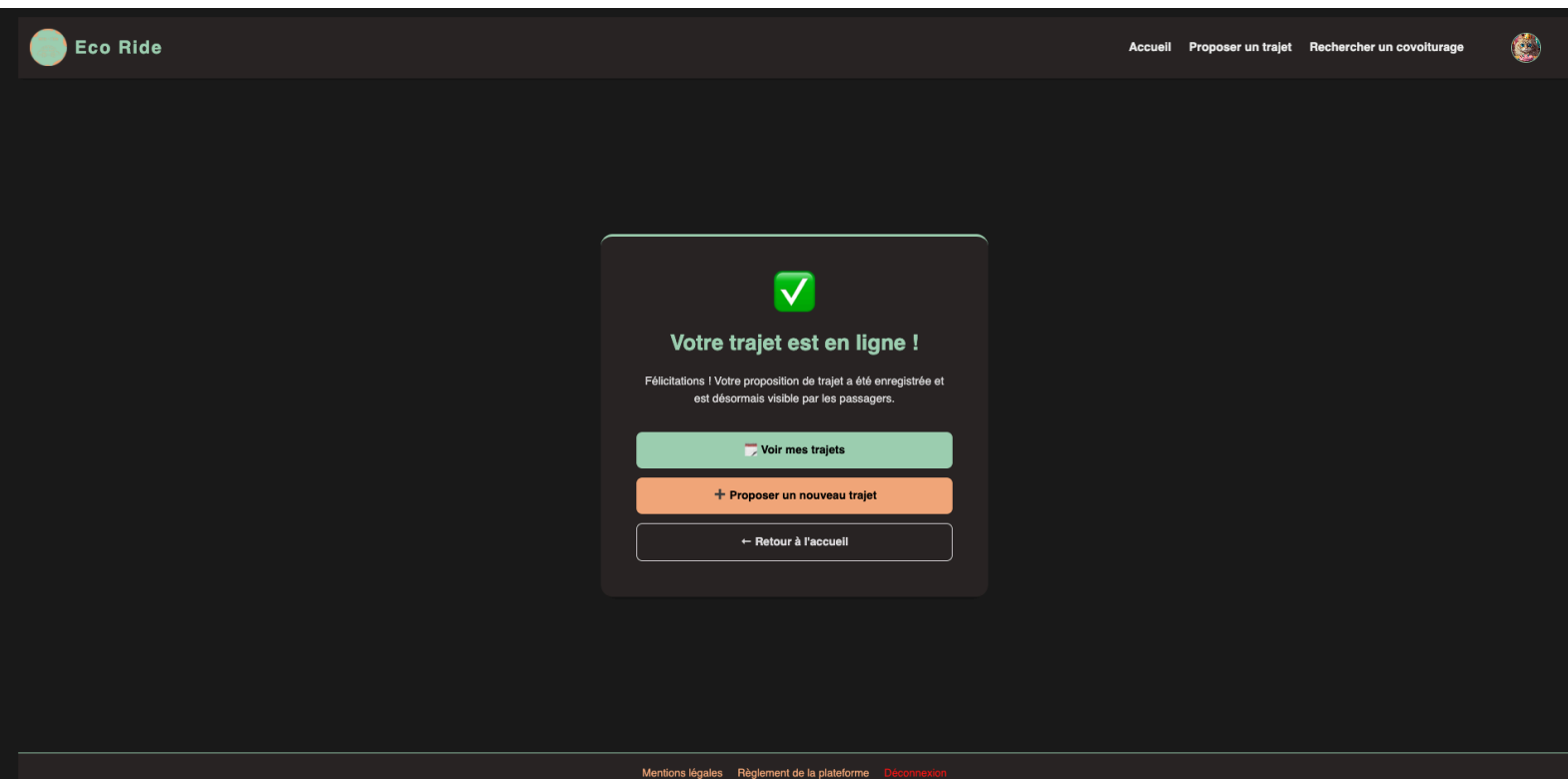
VOS GAINS
3 crédits

POINT DE RENDEZ-VOUS
Départ Carrefour Mérignac soleil,
arrivée gare de Montpon

Confirmer le trajet

[← Modifier les informations](#)

Annexe II.9 : page de confirmation



Annexe II.10 : extrait de code du formulaire sur la page proposer trajet

```
<!-- COLONNE 1 : Départ -->
<div class="trip-card">
  <h3 class="trip-subtitle">📍 Départ</h3>

  <div class="form-group">
    <label for="departure">Adresse de départ *</label>
    <input type="text" id="departure" name="departure" placeholder="Ville ou adresse de départ"
      value="<?= htmlspecialchars($trajet_temp['departure']) ?>">
  </div>

  <label>Arrêts (optionnel)</label>
  <div id="etapes-container">
    <?php foreach ($etapes as $i => $etape): ?>
      <div class="stop-container">
        <input type="text" id="step<?= $i+1 ?>" name="step<?= $i+1 ?>"
          placeholder="Arrêt n°<?= $i+1 ?>"
          value="<?= htmlspecialchars($etape) ?>">
      </div>
    <?php endforeach; ?>
  </div>
  <button type="button" id="add-stop-btn">+ Ajouter un arrêt</button>

  <div class="form-group" style="margin-top: 1rem;">
    <label for="date">Date de départ *</label>
    <input type="date" id="date" name="date"
      value="<?= htmlspecialchars($trajet_temp['date']) ?>">
  </div>

  <div class="form-group">
    <label for="time">Heure de départ *</label>
    <input type="time" id="time" name="time"
      value="<?= htmlspecialchars($trajet_temp['time']) ?>">
  </div>
</div>
```

Annexe II.11 : extrait de code de récupération des données en PHP

```
$_SESSION['trajet_temp'] = [
    'departure' => $_POST['departure'] ?? '',
    'arrival' => $_POST['arrival'] ?? '',
    'date' => $_POST['date'] ?? '',
    'time' => $_POST['time'] ?? '',
    'time_arrivee' => $_POST['time_arrivee'] ?? '',
    'vehicle_used' => $_POST['vehicle_used'] ?? '',
    'places' => $_POST['places'] ?? '',
    'prix' => $_POST['prix'] ?? 2,
    'commentaire' => $_POST['commentaire'] ?? '',
    'etapes' => array_filter($_POST, function($key) {
        return strpos($key, 'step') === 0;
    }, ARRAY_FILTER_USE_KEY)
];

$trajet = $_SESSION['trajet_temp'];
$gains = max(0, (int)$trajet['prix'] - 2);

// Récupérer le nom du véhicule
$vehicule_label = '-';
if (!empty($trajet['vehicle_used'])) {
    $stmt = $bdd->prepare("SELECT marque, modele, couleur FROM vehicules WHERE vehicule_id = ?");
    $stmt->execute([$trajet['vehicle_used']]);
    $vehicule = $stmt->fetch(PDO::FETCH_ASSOC);
    if ($vehicule) {
        $vehicule_label = htmlspecialchars($vehicule['marque'] . ' ' . $vehicule['modele'] . ' ' . $vehicule['couleur']);
    }
}
```

Annexe II.12: extrait de code du JS pour la récupération des arrêts

```
const fromStep2 = sessionStorage.getItem('fromStep2') === 'true';
if (fromStep2) {
  const storedDeparture = sessionStorage.getItem('departure') || '';
  const storedArrival = sessionStorage.getItem('arrival') || '';
  const storedDate = sessionStorage.getItem('date') || '';
  const storedTime = sessionStorage.getItem('time') || '';
  const storedTimeArrivee = sessionStorage.getItem('time_arrivee') || '';
  const storedVehicle = sessionStorage.getItem('vehicle') || '';
  const storedPlaces = sessionStorage.getItem('places') || '';
  const storedComments = sessionStorage.getItem('comments') || '';
  const storedStops = JSON.parse(sessionStorage.getItem('stops') || '[]');

  if (storedDeparture) inputDepart.value = storedDeparture;
  if (storedArrival) inputArrivee.value = storedArrival;
  if (storedDate) inputDate.value = storedDate;
  if (storedTime) inputTime.value = storedTime;
  if (storedTimeArrivee) inputTimeArrivee.value = storedTimeArrivee;
  if (storedVehicle) selectVehicule.value = storedVehicle;
  if (storedPlaces) inputPlaces.value = storedPlaces;
  if (storedComments) inputCommentaire.value = storedComments;
```

Annexe II.13 : extrait du code php pour l'insertion SQL

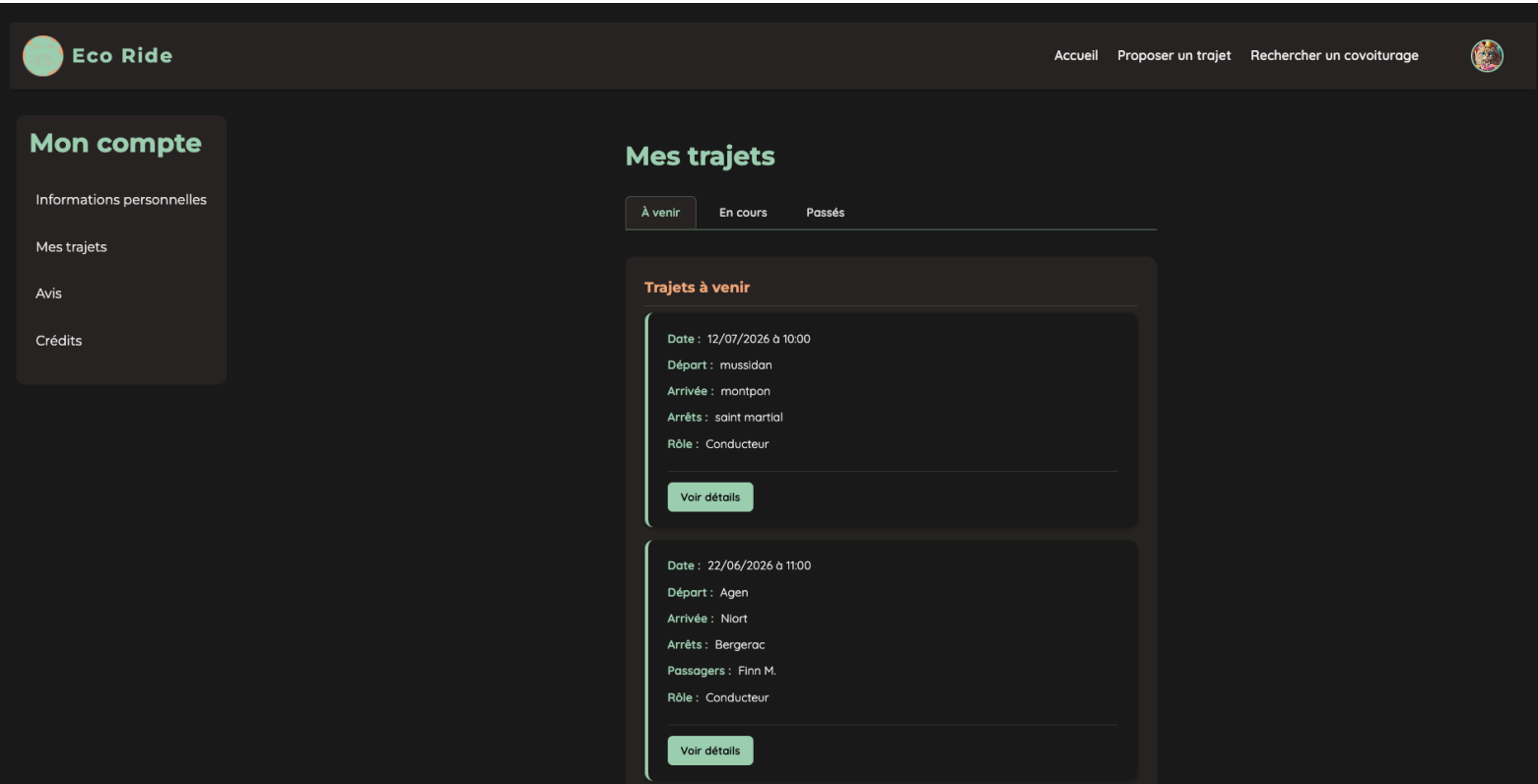
```
$sql = "INSERT INTO trajets
(id_conducteur, depart, arrivee, date_depart, heure_depart, heure_arrivee, vehicule_id, places_disponibles, prix, statut, etapes, commentaire)
VALUES (:id_conducteur, :depart, :arrivee, :date_depart, :heure_depart, :heure_arrivee, :vehicule_id, :places_disponibles, :prix, 'publie', :etapes, :commentaire)";

$stmt = $bdd->prepare($sql);
$stmt->execute([
    ':id_conducteur'    => $id_conducteur,
    ':depart'           => $trajet['departure'],
    ':arrivee'          => $trajet['arrival'],
    ':date_depart'      => $trajet['date'],
    ':heure_depart'     => $trajet['time'],
    ':heure_arrivee'    => $trajet['time_arrivee'] ?? null,
    ':vehicule_id'      => $trajet['vehicle_used'],
    ':places_disponibles' => $trajet['places'],
    ':prix'             => intval($trajet['prix'] ?? 2),
    ':etapes'           => $etapes_str,
    ':commentaire'      => $trajet['commentaire'] ?? ''
]);
```

Annexe II.14 : extrait de code pour l'ajout dynamique des arrêts

```
document.getElementById('add-stop-btn').addEventListener('click', () => {
  const existingStops = document.querySelectorAll('.stop-container');
  const stopNumber = existingStops.length + 2;
  if (stopNumber > 6) { alert("Vous ne pouvez ajouter que 5 arrêts maximum."); return; }
  const stopContainer = document.createElement('div');
  stopContainer.classList.add('stop-container');
  stopContainer.style.marginTop = '10px';
  stopContainer.innerHTML = `
    <label for="step${stopNumber}">Arrêt n°${stopNumber} (optionnel)</label>
    <input type="text" id="step${stopNumber}" name="step${stopNumber}" placeholder="Ajouter un arrêt">
    <button type="button" class="remove-stop">Supprimer</button>
  `;
  const lastStop = existingStops[existingStops.length - 1] || document.getElementById('step1').parentNode;
  lastStop.parentNode.insertBefore(stopContainer, lastStop.nextSibling);
  stopContainer.querySelector('.remove-stop').addEventListener('click', () => {
    stopContainer.remove();
    renumeroterArrets();
  });
});
```

Annexe II.15 : page mes trajets



Annexe II.16 : page détail du trajet

Eco Ride

Accueil

Proposer un trajet

Rechercher un covoiturage

← Retour

Détails du trajet

👤 Conducteur

Clara Soual

★ 4.5 / 5 - 2 avis

🚗 Toyota Corolla - Bleu - Electrique

🌿 Écologique

Avis sur le conducteur

★ Finn M. - 5.0/5

Super trajet, conducteur très sympa et ponctuel !

★ Antoine B. - 4.0/5

Très bon voyage, je recommande !

🗺 Itinéraire

📍 Agen → 📍 Niort

Passant par : Bergerac

📅 DATE

22/06/2026

🕒 DÉPART

11:00

⌚ ARRIVÉE

15:00

👤 PLACES

3 disponible(s)

💰 PRIX

2 crédit(s) / passager

Passagers inscrits

Finn M.

Réserver ce trajet

Mentions légales

Règlement de la plateforme

Déconnexion

ecoride@contact.com

© 2025 ECO RIDE. Tous droits réservés

Annexe II.17 : extrait de code récupération des trajets en PHP

```
// --- Trajets conducteur ---
try {
    $stmt = $bdd->prepare("SELECT *, 'conducteur' AS role FROM trajets WHERE id_conducteur = ?");
    $stmt->execute([$id_utilisateur]);
    $trajetsConducteur = $stmt->fetchAll(PDO::FETCH_ASSOC);
} catch (PDOException $e) {
    die("Erreur PDO (conducteur) : " . htmlspecialchars($e->getMessage()));
}

// --- Trajets passager ---
try {
    $stmt = $bdd->prepare("
        SELECT t.*, 'passager' AS role
        FROM trajets t
        JOIN trajets_passagers tp ON t.id = tp.id_trajet
        WHERE tp.id_passager = ?
    ");
    $stmt->execute([$id_utilisateur]);
    $trajetsPassager = $stmt->fetchAll(PDO::FETCH_ASSOC);
} catch (PDOException $e) {
    die("Erreur PDO (passager) : " . htmlspecialchars($e->getMessage()));
}

$trajets = array_merge($trajetsConducteur, $trajetsPassager);
```

Annexe II.18 : extrait du code tri des trajets selon la date en PHP

```
$trajetsFutur = [];  
$trajetsEnCours = [];  
$trajetsTermine = [];  
$now = new DateTime();  
  
foreach ($trajets as $trajet) {  
    $dateTrajet = new DateTime($trajet['date_depart'] . ' ' . $trajet['heure_depart']);  
  
    if ($trajet['statut'] === 'termine') {  
        $trajetsTermine[] = $trajet;  
    } elseif ($trajet['statut'] === 'en_cours') {  
        $trajetsEnCours[] = $trajet;  
    } elseif ($dateTrajet > $now) {  
        $trajetsFutur[] = $trajet;  
    } else {  
        $trajetsTermine[] = $trajet;  
    }  
}
```

Annexe II.19 : code PHP pour l’affichage d’un trajet

```
function afficherTrajet($trajet, $bdd, $id_utilisateur) {
    $passagers = getPassagers($bdd, $trajet['id']);

    echo '<div class="trip-card">';
    echo '<p><strong>Date :</strong> ' . htmlspecialchars($trajet['date_depart']) . ' à ' . htmlspecialchars($trajet['heure_depart']) . '</p>';
    echo '<p><strong>Départ :</strong> ' . htmlspecialchars($trajet['depart']) . '</p>';
    echo '<p><strong>Arrivée :</strong> ' . htmlspecialchars($trajet['arrivee']) . '</p>';

    if (!empty($trajet['etapes'])) {
        echo '<p><strong>Étapes :</strong> ' . htmlspecialchars($trajet['etapes']) . '</p>';
    }

    if (!empty($passagers)) {
        $listePassagers = array_map(fn($p) => htmlspecialchars($p['nom'] . ' ' . $p['prenom']), $passagers);
        echo '<p><strong>Passagers :</strong> ' . implode(', ', $listePassagers) . '</p>';
    }

    echo '<p><strong>Rôle :</strong> ' . ($trajet['role'] === 'conducteur' ? 'Conducteur' : 'Passager') . '</p>';
    echo '<a href="USR-details-trajet.php?id=' . $trajet['id'] . '" class="btn-details">Voir détails</a>';
}
```

Annexe II.20 : code PHP de vérification du propriétaire

```
/* =====  
VÉRIFICATION PROPRIÉTAIRE  
===== */  
$isOwner = ($_SESSION['user_id'] === $trajet['id_conducteur']);  
  
/* =====  
VÉRIFICATION PASSAGER INSCRIT  
===== */  
$isPassenger = false;  
if (isset($_SESSION['user_id'])) {  
    foreach ($passagers as $p) {  
        if ($_SESSION['user_id'] == $p['id']) {  
            $isPassenger = true;  
            break;  
        }  
    }  
}  
}
```

Annexe II.21 : code du modal de modification de trajet

```
<?php endif; ?>

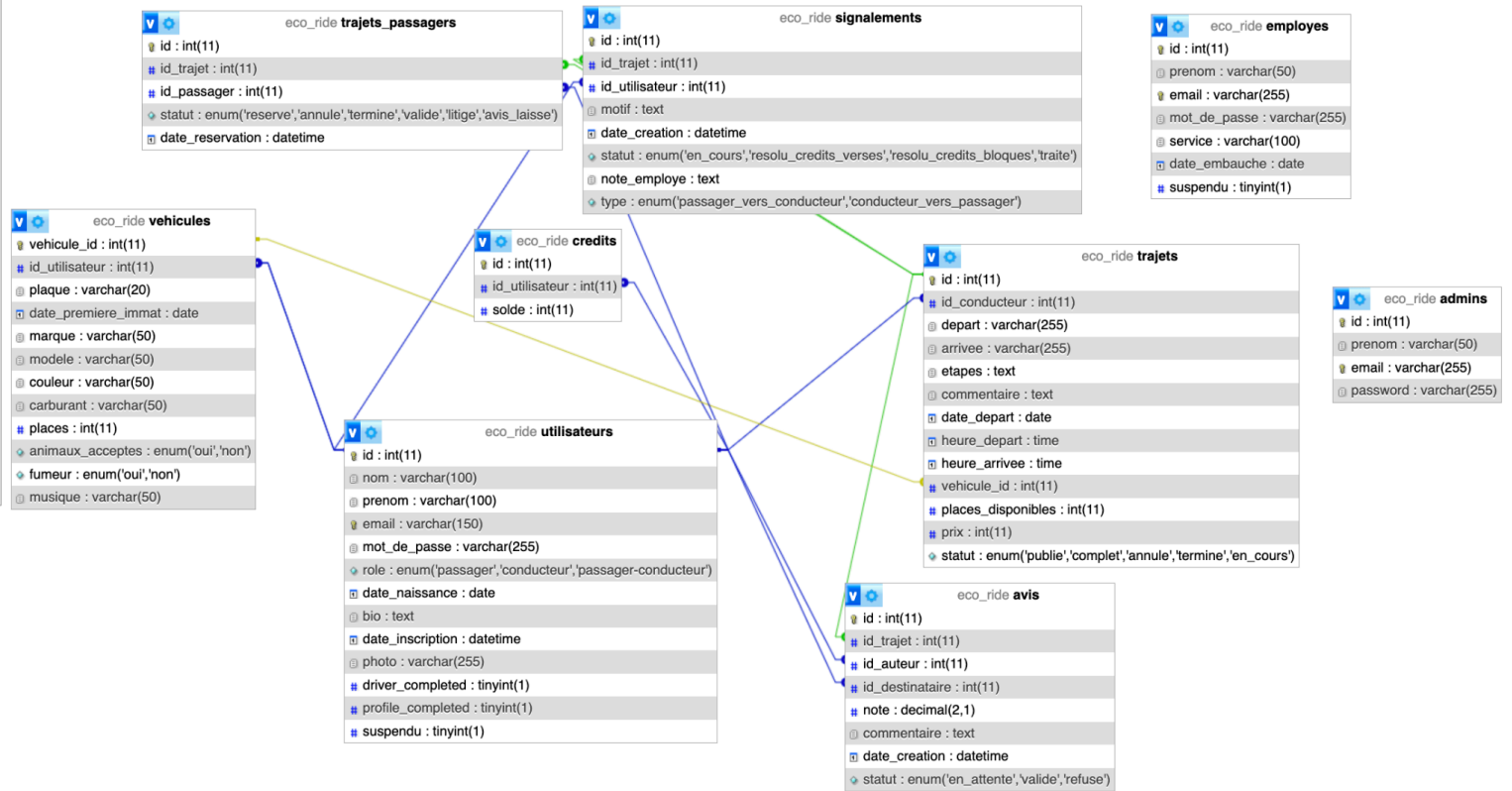
<main>
  <a href="USR-recherche_trajet.php" class="back-arrow">← Retour</a>
  <h1 class="page-title">Détails du trajet</h1>

  <div class="cards-wrapper">

    <!-- CARTE GAUCHE : Conducteur -->
    <section class="card driver-card">
      <h3 class="card-title">👤 Conducteur</h3>
      <div class="driver-main">
        
        <div class="driver-infos">
          <h2><?= htmlspecialchars($trajet['prenom_conducteur'] . ' ' . $trajet['nom_conducteur']) ?></h2>
          <p class="driver-rating">
            <?php if ($nb_avis > 0): ?>
              ★ <?= $note_moyenne ?> / 5 · <?= $nb_avis ?> avis
            <?php else: ?>
              Aucun avis pour le moment
            <?php endif; ?>
          </p>
          <p class="driver-car">
            🚗 <?= htmlspecialchars($trajet['marque'] . ' ' . $trajet['modele']) ?> ·
            <?= htmlspecialchars($trajet['couleur']) ?> ·
            <?= htmlspecialchars($trajet['carburant']) ?>
            <?php if ($eco): ?>
              <span class="eco-badge">🌿 Écologique</span>
            <?php endif; ?>
          </p>
        </div>
      </div>
    </div>

    <div id="driver-reviews" class="driver-reviews">
      <h4>Avis sur le conducteur</h4>
      <?php if (empty($avis)): ?>
        <p class="empty">Aucun avis pour le moment.</p>
      <?php else: ?>
        <?php foreach ($avis as $a): ?>
          <div class="review">
            <p class="review-author">★ <?= htmlspecialchars($a['prenom'] . ' ' . $a['nom']) ?> - <?= $a['note'] ?>/5</p>
            <p class="review-text"><?= htmlspecialchars($a['commentaire']) ?></p>
          </div>
        <?php endforeach; ?>
      <?php endif; ?>
    </div>
  </section>
</div>
```

Annexe III.1 : Diagramme de la base de données



Annexe III.2 : script de création des tables

```
--
-- Structure de la table `admins`
--

CREATE TABLE `admins` (
  `id` int(11) NOT NULL,
  `prenom` varchar(50) NOT NULL,
  `email` varchar(255) NOT NULL,
  `password` varchar(255) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
--
-- Structure de la table `avis`
--

CREATE TABLE `avis` (
  `id` int(11) NOT NULL,
  `id_trajet` int(11) NOT NULL,
  `id_auteur` int(11) NOT NULL,
  `id_destinataire` int(11) NOT NULL,
  `note` decimal(2,1) DEFAULT NULL,
  `commentaire` text DEFAULT NULL,
  `date_creation` datetime DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
--
-- Structure de la table `trajets`
--

CREATE TABLE `trajets` (
  `id` int(11) NOT NULL,
  `id_conducteur` int(11) NOT NULL,
  `depart` varchar(255) NOT NULL,
  `arrivee` varchar(255) NOT NULL,
  `etapes` text DEFAULT NULL,
  `commentaire` text DEFAULT NULL,
  `date_depart` date NOT NULL,
  `heure_depart` time NOT NULL,
  `vehicule_id` int(11) DEFAULT NULL,
  `places_disponibles` int(11) DEFAULT 1,
  `prix` int(11) NOT NULL DEFAULT 2,
  `statut` enum('publie','complet','annule','termine','en_cours') DEFAULT 'publie'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
--
-- Structure de la table `utilisateurs`
--

CREATE TABLE `utilisateurs` (
  `id` int(11) NOT NULL,
  `nom` varchar(100) NOT NULL,
  `prenom` varchar(100) NOT NULL,
  `email` varchar(150) DEFAULT NULL,
  `mot_de_passe` varchar(255) NOT NULL,
  `role` enum('passager','conducteur','passager-conducteur') NOT NULL,
  `date_naissance` date DEFAULT NULL,
  `bio` text DEFAULT NULL,
  `credits` int(11) DEFAULT 0,
  `date_inscription` datetime DEFAULT current_timestamp(),
  `photo` varchar(255) DEFAULT 'default.jpg',
  `driver_completed` tinyint(1) DEFAULT 0,
  `profile_completed` tinyint(1) DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Annexe III.3 : script de données seed

```
--
-- Déchargement des données de la table `admins`
--

INSERT INTO `admins` (`id`, `prenom`, `email`, `password`) VALUES
(1, 'Clara', 'Clara@admin-eco.com', '$2y$10$.pP8pnF6FVJ27.6aV0SoI02M1kSGByHymIcUcWKfRusWB8dwmosJC'),
(2, 'Léon', 'Léon@admin-eco.com', '$2y$10$.pP8pnF6FVJ27.6aV0SoI02M1kSGByHymIcUcWKfRusWB8dwmosJC');

--

INSERT INTO `avis` (`id`, `id_trajet`, `id_auteur`, `id_destinataire`, `note`, `commentaire`, `date_creation`) VALUES
(1, 2, 2, 1, 4.0, 'Trajet confortable, Nino est très respectueux et à l\'écoute de ses passagers. Petit bémol, léger retard à l\'arrivée.', '2026-01-27 10:00:00'),
(2, 2, 3, 1, 5.0, 'Nino est toujours ponctuel et très sympa, je recommande !', '2026-01-27 11:00:00'),
(3, 2, 1, 2, 5.0, 'Finn a été un passager agréable.', '2026-01-27 12:00:00'),
(4, 2, 1, 3, 5.0, 'Antoine a été un passager agréable.', '2026-01-27 12:10:00'),
(5, 3, 5, 2, 5.0, 'Super trajet, Finn est ponctuel et agréable.', '2026-02-08 10:00:00'),
(6, 3, 4, 2, 4.0, 'Finn conduit bien et le trajet est confortable.', '2026-02-08 10:30:00'),
(7, 3, 1, 2, 5.0, 'Tout parfait, trajet agréable.', '2026-02-08 11:00:00'),
(8, 3, 2, 5, 5.0, 'Théo est un passager sympa.', '2026-02-08 12:00:00'),
(9, 3, 2, 1, 5.0, 'Nino est un passager agréable.', '2026-02-08 12:10:00'),
(10, 3, 2, 4, 4.0, 'Nina est un passager correct.', '2026-02-08 12:20:00'),
(11, 4, 4, 3, 3.5, 'Je ne me suis pas sentie à l\'aise.', '2026-03-16 10:00:00'),
(12, 4, 3, 4, 5.0, 'Nina a été une passagère agréable.', '2026-03-16 11:00:00');

--
-- Déchargement des données de la table `trajets`
--

INSERT INTO `trajets` (`id`, `id_conducteur`, `depart`, `arrivee`, `etapes`, `commentaire`, `date_depart`, `heure_depart`, `vehicule_id`, `places_disponibles`, `prix`, `statut`) VALUES
(1, 6, 'Mussidan', 'Perigueux', NULL, NULL, '2026-01-02', '15:00:00', 4, 1, 2, 'termine'),
(2, 1, 'Perigueux', 'Limoges', NULL, NULL, '2026-01-26', '08:00:00', 1, 0, 2, 'termine'),
(3, 2, 'Saint Pierre de Chignac', 'Bordeaux', NULL, NULL, '2026-02-07', '10:00:00', 2, 0, 2, 'termine'),
(4, 3, 'Béziers', 'Bordeaux', NULL, NULL, '2026-03-15', '14:00:00', 3, 0, 2, 'termine'),
(5, 1, 'Bordeaux', 'Bayonne', NULL, NULL, '2026-10-25', '08:30:00', 1, 2, 2, 'publie'),
(6, 1, 'Bayonne', 'Dax', NULL, NULL, '2026-10-30', '17:45:00', 1, 3, 2, 'publie'),
(7, 2, 'Toulouse', 'Auch', NULL, NULL, '2026-09-03', '10:00:00', 2, 3, 2, 'publie'),
(8, 3, 'Agen', 'Pau', NULL, NULL, '2026-09-26', '14:15:00', 3, 0, 2, 'complet'),
(13, 19, 'Bordeaux', 'Biarritz', NULL, '', '2026-03-12', '10:00:00', 15, 2, 2, 'termine'),
(14, 19, 'Bergerac', 'Mussidan', NULL, '', '2026-03-12', '10:00:00', 15, 1, 2, 'termine'),
(39, 19, 'muss', 'montpon', NULL, '', '2027-12-12', '10:00:00', 15, 3, 2, 'publie'),
(42, 19, 'begles', 'gradignan', NULL, '', '2026-04-06', '15:00:00', 15, 2, 3, 'publie');
```

Annexe III.4 : capture d'écran de logs JSON

```
{
  "id": "log_69d50cfaec2151.61823690",
  "date": "2026-04-07 13:56:10",
  "niveau": "INFO",
  "action": "recherche_trajet",
  "message": "Recherche : agen → pau le 2026-09-26 (0 résultat(s))",
  "utilisateur": 19,
  "ip": "172.18.0.1",
  "page": "\\PROJET\\UTILISATEUR\\USR-recherche_trajet.php?departure=agen&destination=pau&date=2026-09-26&passenger=1"
},
{
  "id": "log_69d50ce86524e8.80333043",
  "date": "2026-04-07 13:55:52",
  "niveau": "INFO",
  "action": "recherche_trajet",
  "message": "Recherche : bordeaux → bayonne le 2026-10-25 (1 résultat(s))",
  "utilisateur": 19,
  "ip": "172.18.0.1",
  "page": "\\PROJET\\UTILISATEUR\\USR-recherche_trajet.php?departure=bordeaux&destination=bayonne&date=2026-10-25&passenger=1"
},
{
  "id": "log_69d50ccf789292.63732737",
  "date": "2026-04-07 13:55:27",
  "niveau": "INFO",
  "action": "recherche_trajet",
  "message": "Recherche : bordeaux → bayonne le 2026-10-24 (0 résultat(s))",
  "utilisateur": 19,
  "ip": "172.18.0.1",
  "page": "\\PROJET\\UTILISATEUR\\USR-recherche_trajet.php?departure=bordeaux&destination=bayonne&date=2026-10-24&passenger=1"
},
{
```

Annexe III.5 : page admin logs

Espace Administrateur

Eco Ride

Nom : Léon
Email : leon@admin-eco.com

Tableau de bord

Comptes

Employés

Utilisateurs

Statistiques

Graphiques & crédits

Supervision

Logs

Logs

121 entrées

121
TOTAL

107
INFO

14
WARNING

NIVEAU

Tous

ACTION

ex: recherche_trajet

Filtrer

Réinitialiser

Date	Niveau	Action	Message	Utilisateur	IP	Page
2026-06-26 16:15:23	INFO	recherche_trajet	Recherche : agen → pau le 2026-09-26 (1 résultat(s))	#19	172.18.0.1	/UTILISATEUR/USR-recherche_trajet.php?departure=agen&destination=pau&date=2026-09-26&passenger=1
			Recherche : agen → pau			/UTILISATEUR/USR-recherche

Annexe III.6 : table utilisateurs

id	nom	prenom	email	mot_de_passe	role	date_naissance	bio	date_inscription	photo
1	C.	Nino	nino@example.com	\$2y\$12\$/4Y0ccznDIh/djcAGMYb.QbNVtU1u2uOiXdvDwrveE...	passager-conducteur	1998-01-01	NULL	2025-11-06 16:57:52	nino.jpg
2	M.	Finn	finn@example.com	\$2y\$12\$/4Y0ccznDIh/djcAGMYb.QbNVtU1u2uOiXdvDwrveE...	passager-conducteur	1990-01-01	NULL	2025-11-06 16:57:52	finn.png
3	B.	Antoine	antoine@example.com	\$2y\$12\$/4Y0ccznDIh/djcAGMYb.QbNVtU1u2uOiXdvDwrveE...	passager-conducteur	1983-01-01	NULL	2025-11-06 16:57:52	antoine.jpg
4	R.	Nina	nina@example.com	\$2y\$12\$/4Y0ccznDIh/djcAGMYb.QbNVtU1u2uOiXdvDwrveE...	passager	2002-01-01	NULL	2025-11-06 16:57:52	nina.jpg
5	K.	Théo	theo@example.com	\$2y\$12\$/4Y0ccznDIh/djcAGMYb.QbNVtU1u2uOiXdvDwrveE...	passager	1995-01-01	NULL	2025-11-06 16:57:52	theo.jpg
6	S.	Jeanne	jeanne@example.com	\$2y\$12\$/4Y0ccznDIh/djcAGMYb.QbNVtU1u2uOiXdvDwrveE...	conducteur	1963-01-01	NULL	2025-11-06 16:57:52	jeanne.jpg

Annexe III.7 : logs de connexion dans le fichier JSON

```
{
  "id": "log_6a3e9f24ccaa78.95995670",
  "date": "2026-06-26 15:47:48",
  "niveau": "INFO",
  "action": "connexion",
  "message": "Connexion réussie : clara.soual@example.com",
  "utilisateur": 19,
  "ip": "172.18.0.1",
  "page": "\PHP\login.php"
},
{
  "id": "log_6a3e9ea9755701.65456620",
  "date": "2026-06-26 15:45:45",
  "niveau": "WARNING",
  "action": "connexion_echouee",
  "message": "Mot de passe incorrect pour : clara.soual@example.com",
  "utilisateur": 19,
  "ip": "172.18.0.1",
  "page": "\PHP\login.php"
},
```

Annexe IV.1 : extrait docker-compose.yml

```
services:

  # Serveur Apache + PHP
  php:
    build: .
    container_name: ecoride_php
    ports:
      - "8080:80"
    volumes:
      - ./:/var/www/html
    depends_on:
      - mysql
    environment:
      - DB_HOST=mysql
      - DB_NAME=eco_ride
      - DB_USER=root
      - DB_PASS=root

  # Base de données MySQL
  mysql:
    image: mysql:8.0
    container_name: ecoride_mysql
    ports:
      - "3307:3306"
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: eco_ride
    volumes:
      - mysql_data:/var/lib/mysql

  # phpMyAdmin
  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    container_name: ecoride_phpmyadmin
    ports:
      - "8081:80"
    environment:
      PMA_HOST: mysql
      MYSQL_ROOT_PASSWORD: root
    depends_on:
      - mysql

volumes:
  mysql_data:
```

Annexe IV.2 : screenshot du site accessible sur le localhost 8080

The image shows a Docker Desktop interface with the **eco_ride** service running. The service is a PHP application with a MySQL database, accessible on port 8080. The logs show the MySQL server starting and listening on port 3306.

Docker Desktop Interface:

- Service:** eco_ride (C:\xampp\htdocs\eco_ride)
- Containers:** php (eco_ride-php 8080:80), phpmyadmin (phpmyadmin/phpmyadmin 8081:80), mysql (mysql:8.0 3307:3306)
- Logs:** removed in a future release. Please use SET GLOBAL host_cache_size=0 instead. 2026-04-27T12:09:32.688060Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.0.45) starting as process 1. 2026-04-27T12:09:32.698946Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started. 2026-04-27T12:09:33.083131Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended. 2026-04-27T12:09:33.316135Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed. 2026-04-27T12:09:33.316173Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for this channel. 2026-04-27T12:09:33.319185Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different directory. 2026-04-27T12:09:33.343027Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port: 33060, socket: /var/run/mysqld/mysqlx.sock 2026-04-27T12:09:33.343330Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.0.45' socket: '/var/run/mysqld/mysql.sock' port: 3306 MySQL Community Server - GPL.

Browser View (localhost:8080/PROJET/UTILISATEUR/USR-index.php):

- Header:** Eco Ride (Accueil, Proposer un trajet, Rechercher un covoiturage)
- Image:** Ensemble, roulons vers un futur plus vert
- Form:** Je pars de ... (Ville de départ), Je vais à ... (Ville d'arrivée), Date (jj/mm/aaaa), Passagers (1)
- Footer:** José Marceau

←↻localhost:8081/index.php?route=/database/structure&db=eco_ride

phpMyAdmin

Server: mysqlBase de données: eco_ride

StructureSQLRechercherRequêteExporterImporterOpérationsPrivilegesProcédures stockéesÉvènementsDéclencheursConcepteur

RécentesPréférences

Nouvelle base de donnéeseco_ride

Nouvelle table

adminsaviscreditsemployessignalementstrajetspassagersutilisateursvehicules

information_schema

mysql

performance_schema

sys

Filtres

Contenant le mot :

Table	Action	Lignes	Type	Interclassement	Taille	Perte
☐ admins	Parcourir Structure Rechercher Insérer Vider Supprimer	2	InnoDB	utf8mb4_general_ci	32,0 kio	-
☐ avis	Parcourir Structure Rechercher Insérer Vider Supprimer	14	InnoDB	utf8mb4_general_ci	64,0 kio	-
☐ credits	Parcourir Structure Rechercher Insérer Vider Supprimer	2	InnoDB	utf8mb4_general_ci	32,0 kio	-
☐ employes	Parcourir Structure Rechercher Insérer Vider Supprimer	3	InnoDB	utf8mb4_general_ci	32,0 kio	-
☐ signalements	Parcourir Structure Rechercher Insérer Vider Supprimer	2	InnoDB	utf8mb4_general_ci	48,0 kio	-
☐ trajets	Parcourir Structure Rechercher Insérer Vider Supprimer	14	InnoDB	utf8mb4_general_ci	48,0 kio	-
☐ trajets_passagers	Parcourir Structure Rechercher Insérer Vider Supprimer	12	InnoDB	utf8mb4_general_ci	48,0 kio	-
☐ utilisateurs	Parcourir Structure Rechercher Insérer Vider Supprimer	9	InnoDB	utf8mb4_general_ci	32,0 kio	-
☐ vehicules	Parcourir Structure Rechercher Insérer Vider Supprimer	5	InnoDB	utf8mb4_general_ci	32,0 kio	-
9 tables	Somme	63	InnoDB	utf8mb4_0900_ai_ci	368,0 kio	0 o

↑

☐ Tout cocher

Avec la sélection :

Imprimer

Dictionnaire de données

Créer une nouvelle table

Nom de table

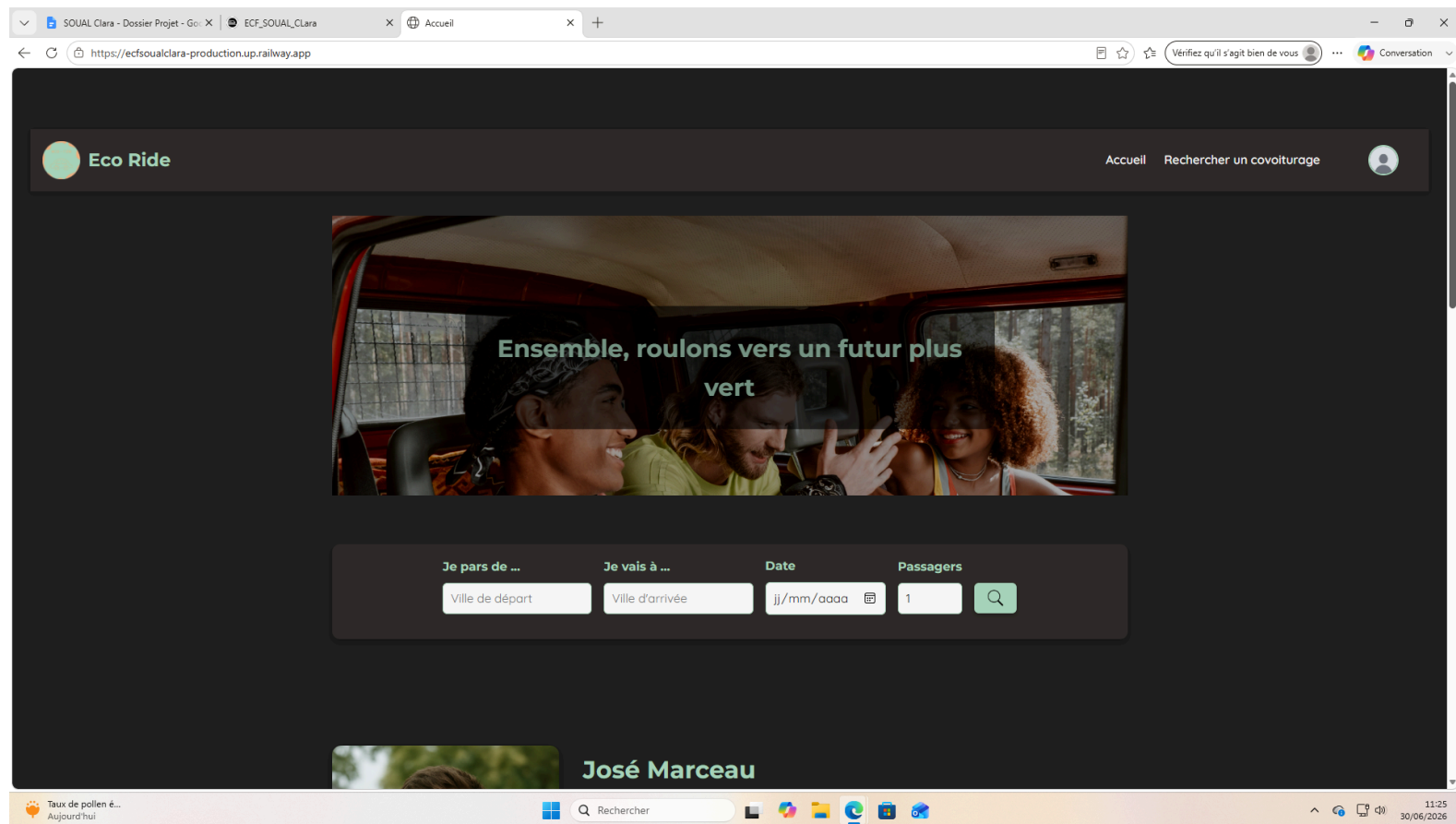
Nombre de colonnes

4

Créer

Console de requêtes SQL

Annexe IV.3 : screenshot du projet déployé sur Railway



Eco Ride — SOUAL Clara

Projet ECF — Session Juin/Juillet 2026 — Titre Professionnel DWM

Présentation du projet

Eco Ride est une plateforme de covoiturage éco-responsable permettant aux utilisateurs de proposer ou réserver des trajets, tout en limitant l'impact environnemental des déplacements.

Le projet est un site full-stack PHP avec une base de données relationnelle MySQL, un système de logs NoSQL (JSON), et un déploiement en production via Docker et Railway.

Rôles disponibles : Visiteur · Utilisateur (Passager / Conducteur / Passager-Conducteur) · Employé · Administrateur

Comptes de démonstration

Rôle	Email	Mot de passe
Administrateur	leon@admin-eco.com	Mdp12345!
Employé	balthazar@emp-eco.com	Mdp12345!
Utilisateur	nino@example.com	Mdp12345!

Lancer le projet en local avec XAMPP

Annexe IV.5 : screenshot de la base de données sur Railway

The screenshot displays the Railway MySQL database interface. The top navigation bar includes the project name 'EcoRide' and the environment 'Production'. The main panel is titled 'MySQL' and features a sidebar with a grid icon, a list icon, and a settings icon. The 'Base de données' tab is selected, showing a 'Données' section with a 'Connectez-vous' button. Below this, the 'Tableaux' section displays a search bar with the query 'SELECT * FROM admins' and a '+ Nouvelle table' button. A grid of table icons is shown, including 'administrateurs', 'avis', 'Crédits', 'Employés', 'signalements', 'Trajets', 'trajets_passagers', 'Utilisateurs', and 'Véhicules'. The bottom status bar shows the system clock at 11:26 on 30/06/2026 and a search bar with the text 'Rechercher'.

Un développeur que vous connaissez a-t-il perdu la paix ? C'est vous le développeur ? [Trouve une voie plus pacifique.](#)

EcoRide / Production

MySQL

Déploiements Base de données Remplaçants Variables Métriques Console Décors

Données Statistiques Configuration [Connectez-vous](#)

Tableaux [+ Nouvelle table](#)

SELECT * FROM admins

administrateurs avis Crédits Employés signalements Trajets

trajets_passagers Utilisateurs Véhicules

Taux de pollen é... Aujourd'hui

Rechercher

11:26 30/06/2026